

TdxQuant 简介

TdxQuant是由深圳市财富趋势科技股份有限公司研发的专业量化投研平台，专注于为国内量化投资者提供从策略研究到投资决策的全流程解决方案。平台以高效、简洁为核心设计理念，致力于降低量化交易门槛，提升策略开发与执行的效率。

依托通达信近三十余年在金融科技领域的深厚积累，TdxQuant集成了完备的实时和历史行情数据、金融数据库及稳定的交易系统基础设施，为策略的研发、回测、验证和执行提供了坚实可靠的技术支持。

平台采用分层化、模块化的服务体系，可灵活适配从高校学生、独立研究者、个人投资者到专业机构等不同用户的需求，实现从策略构思到交易落地的无缝衔接。

TdxQuant 服务介绍

TdxQuant 是一套基于通达信金融终端构建的 Python 量化策略运行框架。该框架通过 API 接口形式，为策略交易提供所需的行情数据获取与交易指令执行功能。

运行环境要求

TdxQuant 支持 64 位 Python 3.7、3.8、3.9、3.10、3.11、3.12、3.13等版本，系统会自动适配当前 Python 版本，建议使用3.13版本。

请注意：运行 TdxQuant 程序前，需预先启动支持TQ策略功能的 **通达信金融终端、专业研究版等版本**。

核心运行逻辑

TdxQuant 以 **tqcenter** 行情模块为核心，专注于为量化交易者提供高效、直接的数据服务，主要包含以下内容：

- **行情数据：**实时与历史的快照、K 线、分笔（Tick）数据
- **基本面数据：**除权除息、基本财务、专业财务、股票交易数据、市场数据等
- **新股和合约等信息：**标的基础信息、可转债、新股申购等
- **分类数据：**市场类型、行业分类、自定义板块等

核心应用场景

TdxQuant提供覆盖量化投研全流程的核心功能模块，主要应用场景包括：

1. 策略研发与历史回测

平台提供“即用型”标准化数据。所有历史与实时数据均在服务端完成清洗、对齐，并预加载至客户端。支持用户快速获取指定时间维度的历史数据，并进行策略信号计算与回测分析。既提供复权因子，也提供各种类型的复权后的数据。

2. 实时监控与信号预警

支持实时行情数据订阅，用户可基于自定义的指标与因子模型进行在线计算。当预设条件触发时，系统通过信号接口实时推送预警信息至客户端，助力研究者及时捕捉市场动态与交易机会。

3. 交易模拟与实盘执行

平台构建了完整的策略交易闭环，提供模拟交易、券商实盘等两种执行环境：

- **模拟交易：**在仿真市场环境中，使用实时行情数据对策略进行持续跟踪与验证，评估其实际表现，全程无资金风险。
- **实盘交易：**通过稳定的交易总线，安全对接券商报盘系统，实现策略信号的自动化、高可靠性下单与交易管理。

量化交易的核心价值

1. 利用历史数据高效验证策略，提升研究效率数百倍

在验证交易策略时，历史回测是评估其有效性的关键环节，但传统人工方式难以处理海量数据与复杂计算。量化交易可在几分钟内完成一次全面回测，快速获得统计验证结果，极大提升了策略研发的迭代效率。

2. 实时捕捉基于概率的获胜机会



错失。它能够综合考量选股、择时、资产配置与风险管理，构建并执行具有较大概率的投资组合，追求收益最大化。

3. 实现科学、客观的投资决策

与传统主观投资不同，量化交易将投资理念、经验甚至市场直觉转化为严谨的数学模型。通过系统化的信号生成与执行机制，有效克服人性中的情绪偏差，使投资决策过程更具纪律性、可重复性与可优化性。

量化交易的工具挑战

工欲善其事，必先利其器。 对于个人投资者而言，独立搭建一套完整的量化交易体系，复杂繁琐，涉及数据、系统、策略等多层面的巨大投入。

一、需要准确、全面的金融数据基础

量化交易依赖于高质量的历史与实时数据，包括行情、财务、宏观及基本面数据等。构建和维护这样一个数据仓库，不仅需要持续的数据采购、清洗、更新与运维成本，还需在数据存储、访问速度与系统稳定性方面进行深入的技术投入。

二、需要易用、可靠的量化交易系统

一个成熟的量化平台需要支持多样的策略开发语言、具备高速的回测与模拟引擎、提供科学的策略评估体系，并为实盘交易提供全方位的保障。过往，研究者往往需要兼具复杂的金融数据知识与工程构建能力。如今，TdxQuant让您只需专注于策略逻辑本身，其余复杂工作交给平台。

TdxQuant的核心优势

TdxQuant是一款集金融数据与策略投研工具于一体的量化平台，结构清晰，简洁易上手，数据获取快捷，算法资源丰富。我们的目标是为用户提供“开箱即用”的完整解决方案。

1. 全方位保障策略安全与自主



2. 大幅降低量化交易门槛

- 提供高质量、高精度、快速接入的金融元数据
- 支持多种策略类型的便捷编写、回测、模拟与实盘

3. 助力构建专业量化成长路径

- 通过"投资学院"系统学习量化交易相关知识体系
- 通过"宽客社区"交流心得、解答疑惑
- 全程助力用户从入门到精通，成为专业的量化投资者

安装Python及VSCode等开发环境

1.安装 Python 环境

安装Python：建议使用Python3.7及以上版本

1.1 下载地址：[Python官网](#)

特别提示：安装时候务必勾选Add Python to PATH（将Python添加到环境变量）

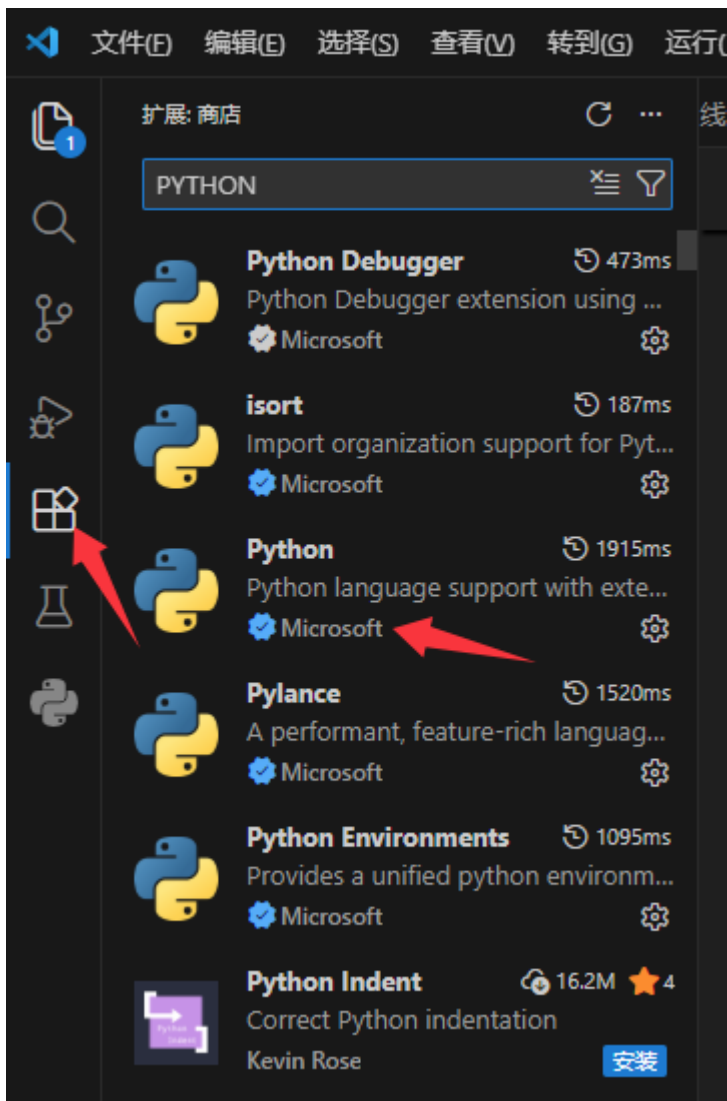
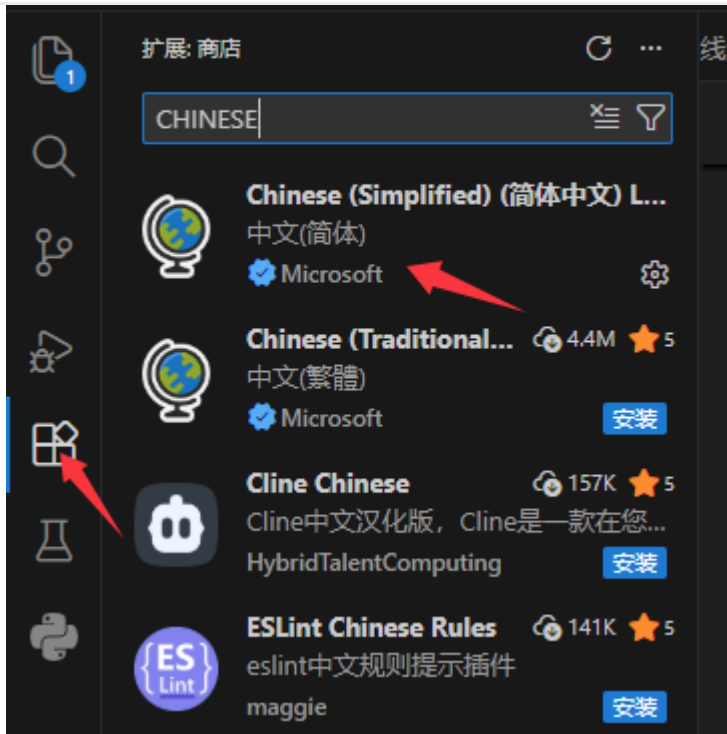
2.安装IDE：建议使用VSCode或PyCharm

2.1 下载地址：[Visual Studio Code官网](#)

2.2 安装 Python 插件（Extensions）

VSCode安装好后，在VSCode终端-扩展-输入下文，分别添加相关扩展：

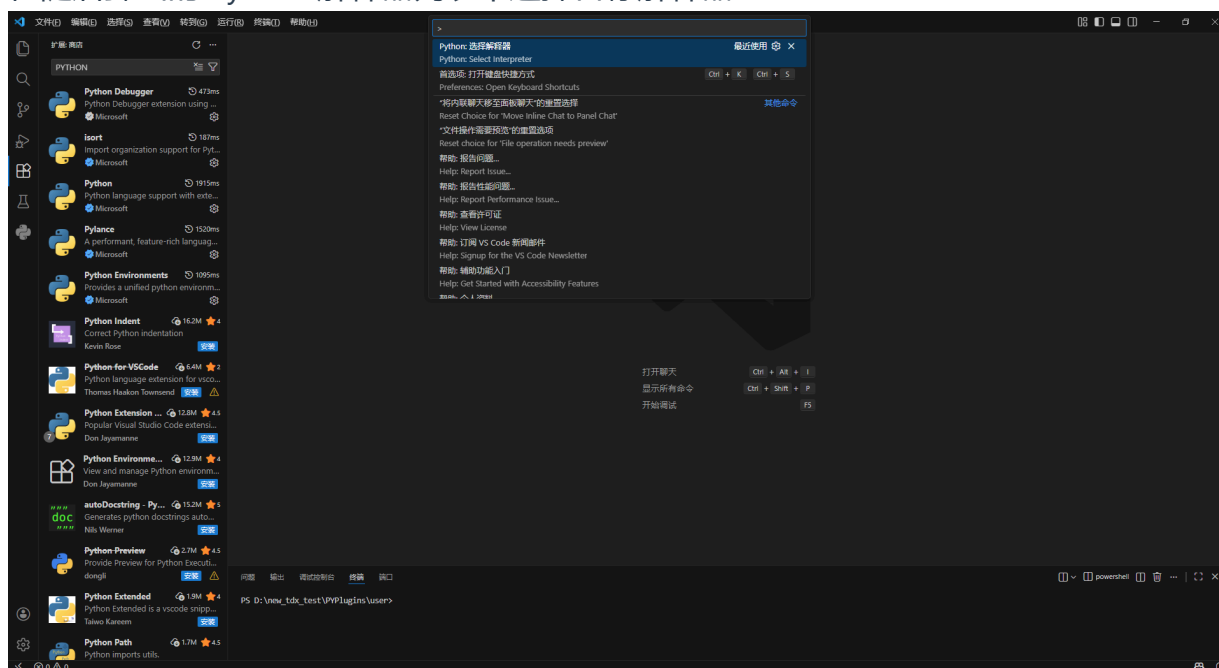
- 简体中文





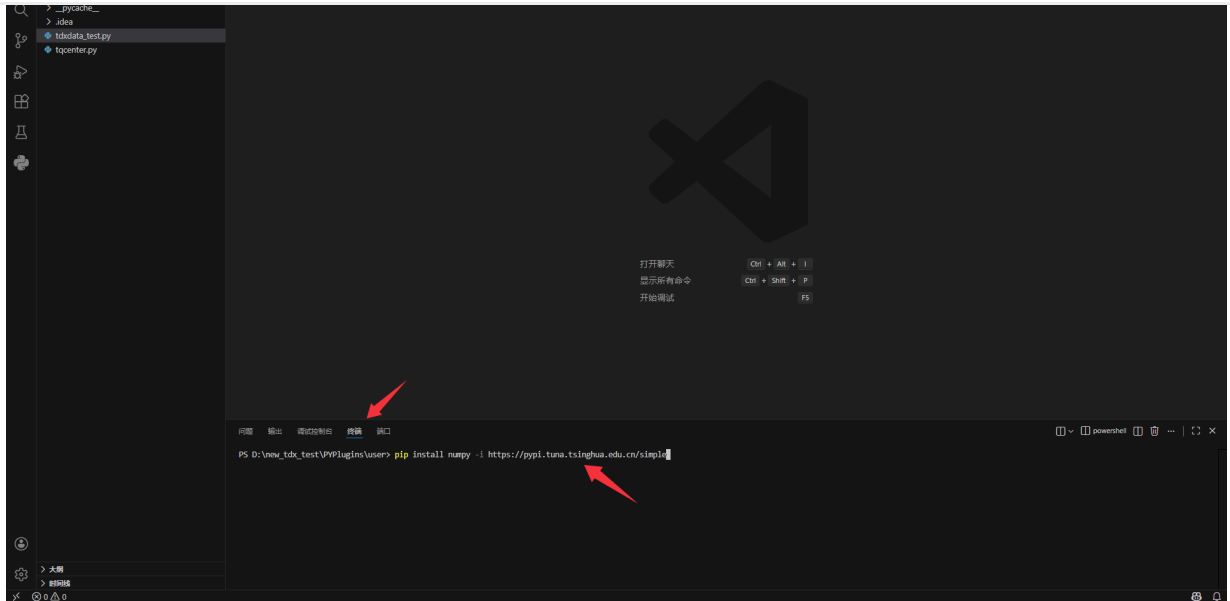
2.3 选择Python解释器：选择python3.13安装路径的exe

- 使用 Ctrl+Shift+P 快捷键打开 command palette 窗口
- 输入关键字 `python select` 并找到 `Python: Select Interpreter` 一项，点击该项并在随后弹出的 Python 解释器列表中选择目标解释器：

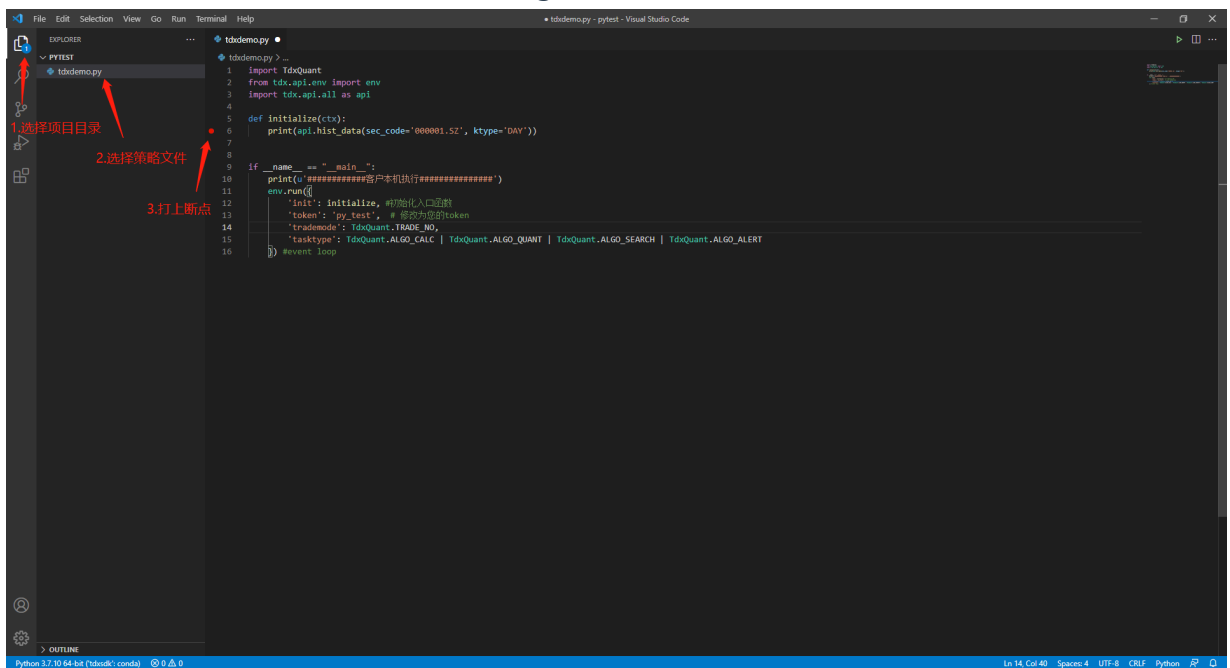


2.4 在VSCode终端-扩展-分别输入下文，常用库建议安装：

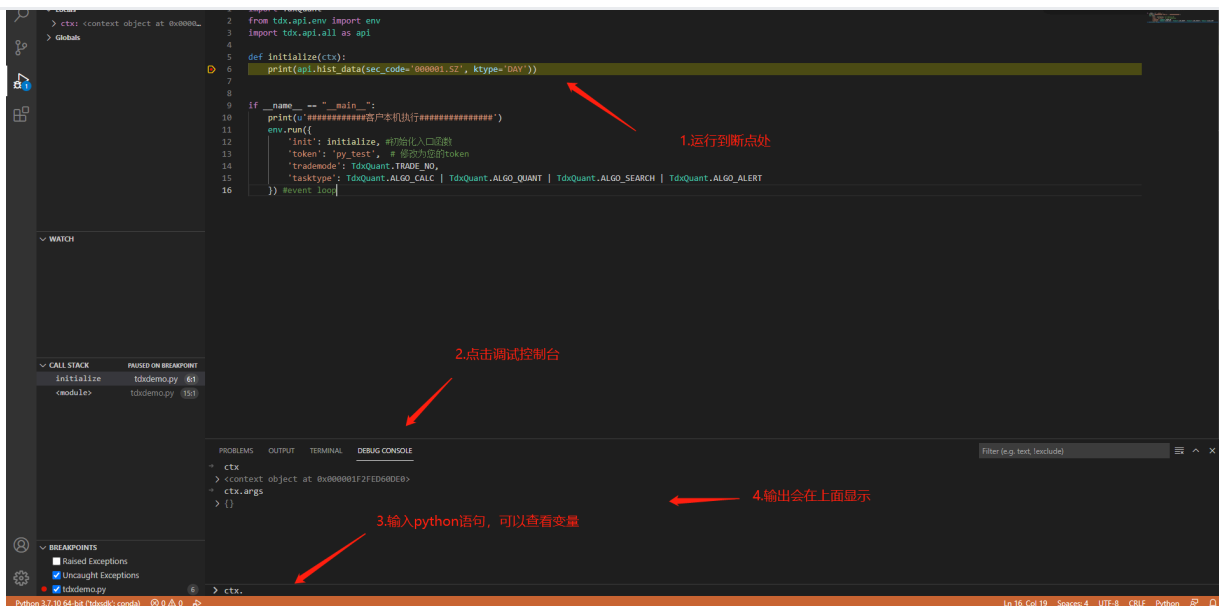
- `pip install numpy -i https://pypi.tuna.tsinghua.edu.cn/simple`
- `pip install pandas -i https://pypi.tuna.tsinghua.edu.cn/simple`
- `pip install backtrader -i https://pypi.tuna.tsinghua.edu.cn/simple`



- 在需要的文件上打上断点，然后 debug（按F5）运行。



- 显示选择调试配置-“ Python文件 ”，调试打开的 Python 文件。



3.用户py的文件位置

在策略管理器界面，点击[文件位置]。

用户的py文件一般在客户端PYPlugins下面的user目录下面。py运行过程的生成的文件一般在PYPlugins下面的data和file目录下。

← [版本更新说明](#)

[安装通达信终端并获取数据](#) →



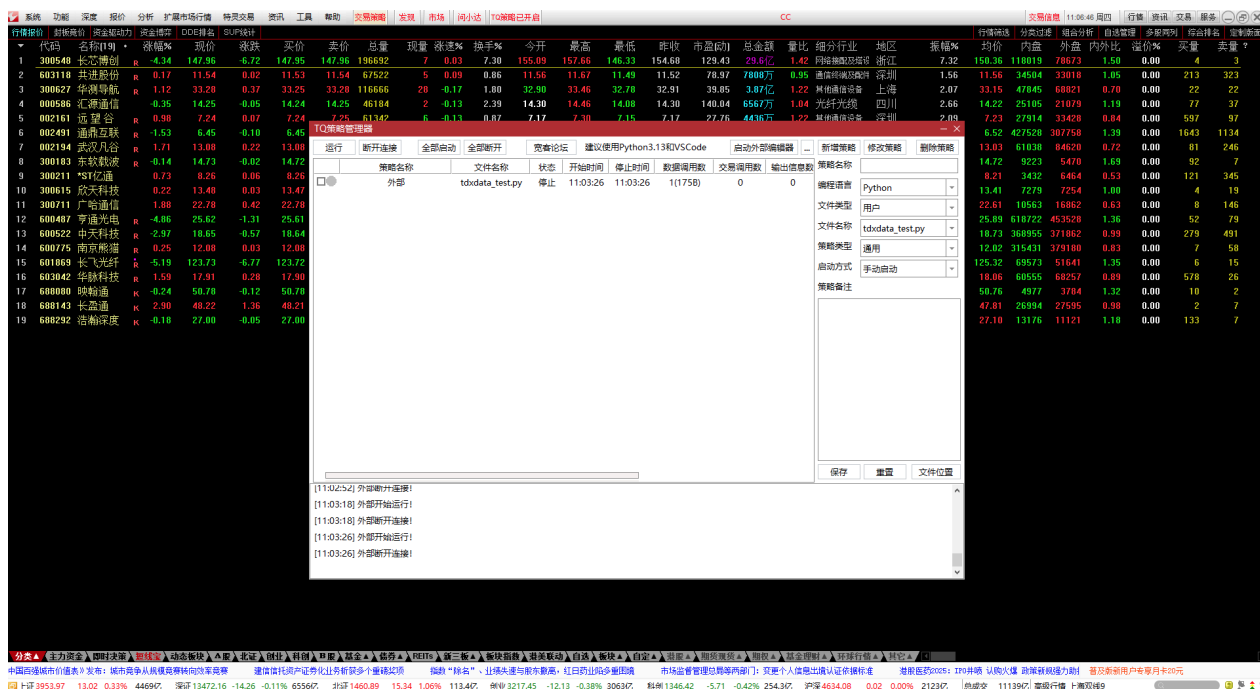
1.安装通达信金融终端内测版

1.1下载地址：[通达信金融终端内测版](https://www.tdx.com.cn/soft.html) 持续更新中

或 正式版 下载通达信专业研究版

<https://www.tdx.com.cn/soft.html>

1.2安装后登录通达信金融终端



1.3 打开菜单 系统-盘后数据下载 ,进行日线和分钟线数据下载



☒ 日线,实时行情和日线统计数据

2025-12-19

▼

~

2025-12-25

▼

☐ 下载所有AB股类品种的日线数据

下载数据覆盖本地原有数据。本地选股、报表分析、日线以上周期本地复权等需要使用这些数据。网络慢时请酌情下载，选择较少品种和较短时间段。

交易日在15:45后才能下载到当天沪深京数据，其它市场行情数据请在相应市场收盘以后下载。

日线以上周期数据通过下载日线数据生成，5倍数的分钟线数据通过5分钟线生成，其它分钟线数据通过1分钟线生成。

较长日线段没有下载时,可点击此处获取日线完整包

品种代码	品种名称
🏦 600000	浦发银行
🏦 000001	平安银行

添加品种

移出品种

清空品种

下载完毕.

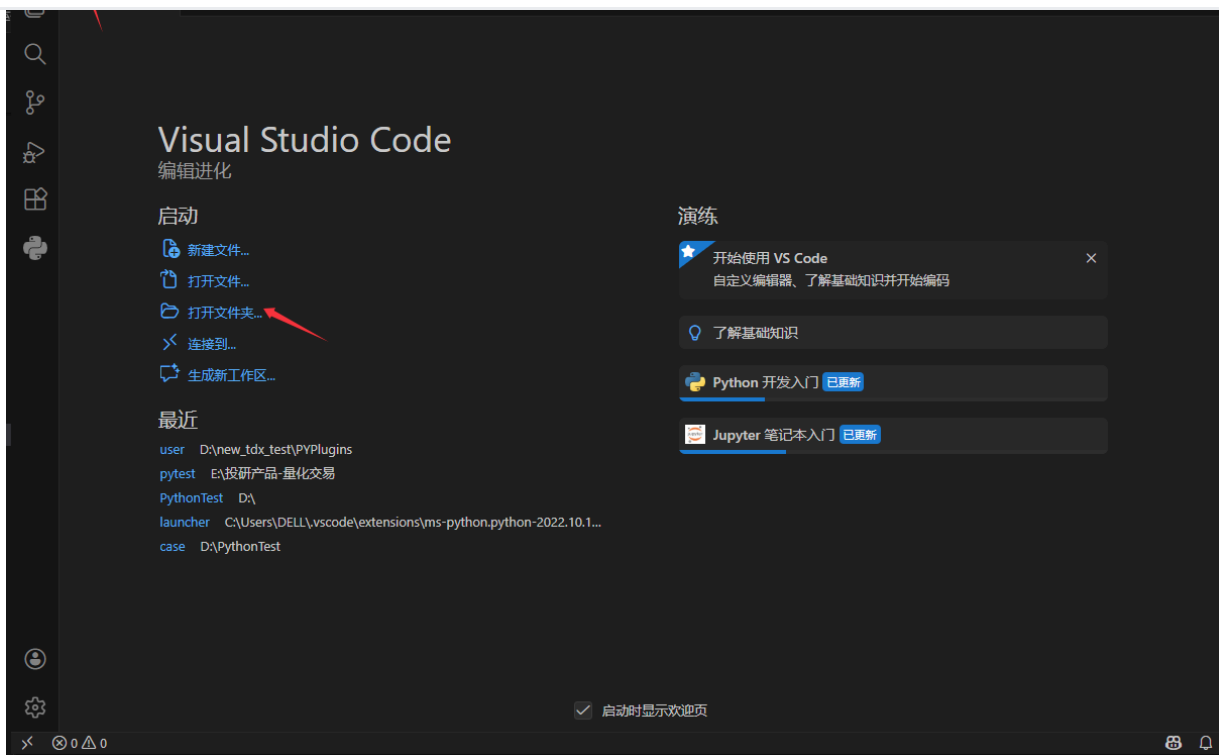
开始下载

关闭

2. 使用VSCode本地编辑和运行py文件

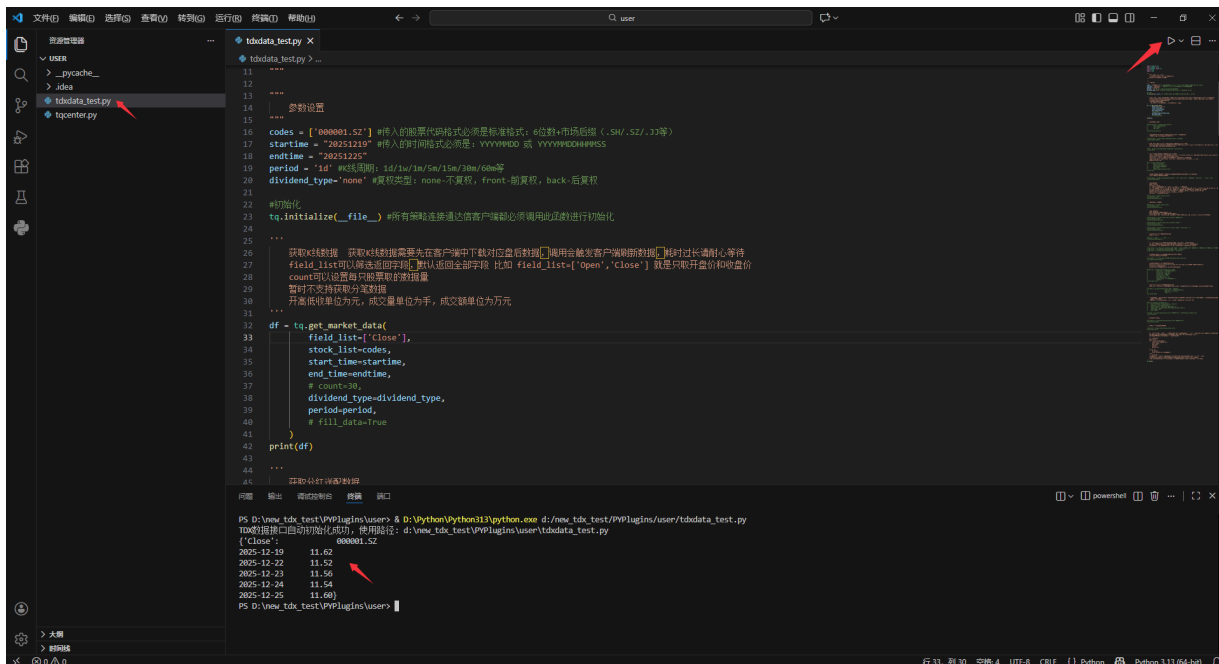
2.1 使用 VSCode 运行

2.1.1 打开py文件



2.1.2 运行py文件

- 在VSCode中打开通达信终端目录 `.../PYPlugins/user` 文件夹，运行tdxdata_test.py文件。

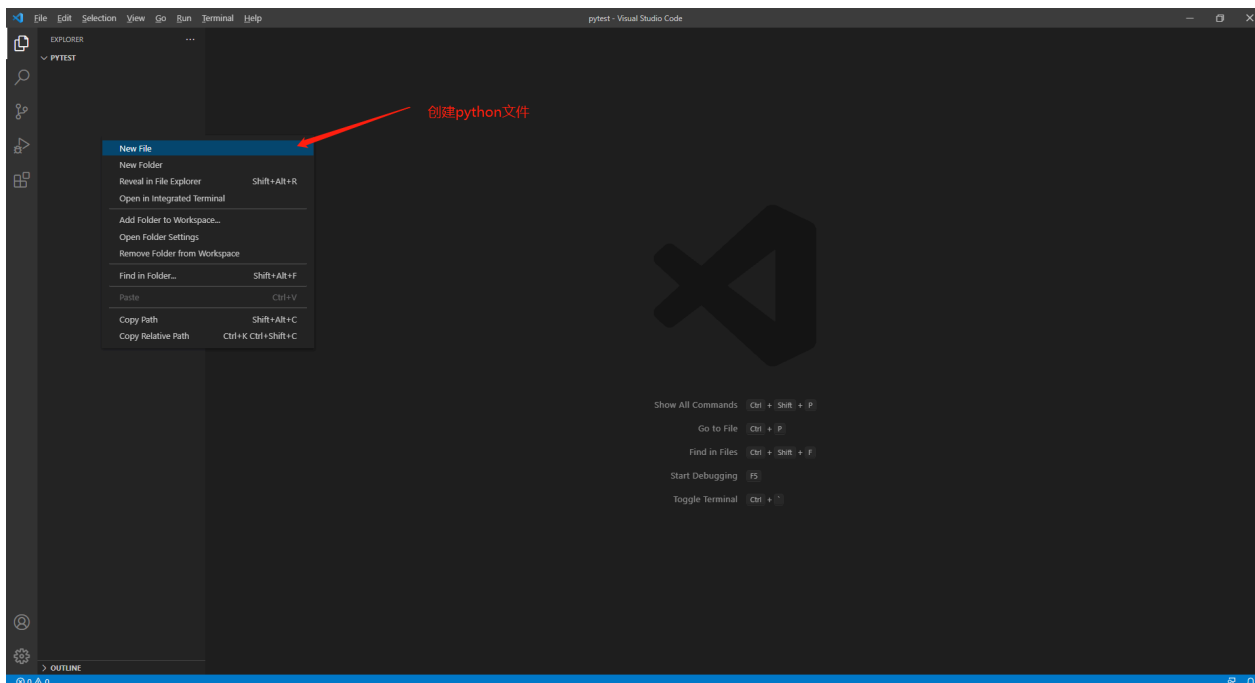


注意：客户端安装目录下面的 `.../PYPlugins/user` 文件夹中的 `tqcenter.py` 是最主要的TQData支撑文件，请勿修改或删除，否则需要重新下载。

2.2 使用 VSCode 编辑新文件并运行

2.2.1 新建py文件

在打开的文件夹中鼠标右键创建新的".py" python 文件，文件名例如tdxdemo.py。



2.2.2 编辑py文件

```
1
2  # 使用tqcenter的API函数查看平安银行日线数据示例
3  from tqcenter import tq
4
5  #初始化
6  tq.initialize(__file__) #所有策略连接通达信客户端都必须调用此函数进行初始化
7
8  #获取平安银行日线前复权收盘数据
9  df = tq.get_market_data(
10      field_list = ['Close'],
11      stock_list = ["000001.SZ"],
12      start_time = '20251219',
13      end_time = '20251225',
14      dividend_type='front',
15      period='1d',
16  )
```



运行结果如图：

```
1 from tqcenter import tq
2
3 #初始化
4 tq.initialize(_file_) #所有策略连接通过该客户端必须调用此函数进行初始化
5
6 #获取平安银行日线数据并返回数据
7 df = tq.get_market_data(
8     field_list = ['Close'],
9     stock_list = ["000001.SZ"],
10    start_time = "20251219",
11    end_time = "20251225",
12    dividend_type="front",
13    period="1d",
14)
15 print(df)
16
17
```

问题 输出 调试控制 终端 窗口

```
PS D:\new_test\PPV\plugins\user> & D:\Python\Python313\python.exe d:\new_test\PPV\plugins\user\tddemo.py
TQ策略接口(自动)初始化成功，使用路径: d:\new_test\PPV\plugins\user\tddemo.py
{'Close':      000001.SZ
2025-12-19      11.62
2025-12-22      11.52
2025-12-23      11.56
2025-12-24      11.54
2025-12-25      11.56}
PS D:\new_test\PPV\plugins\user>
```

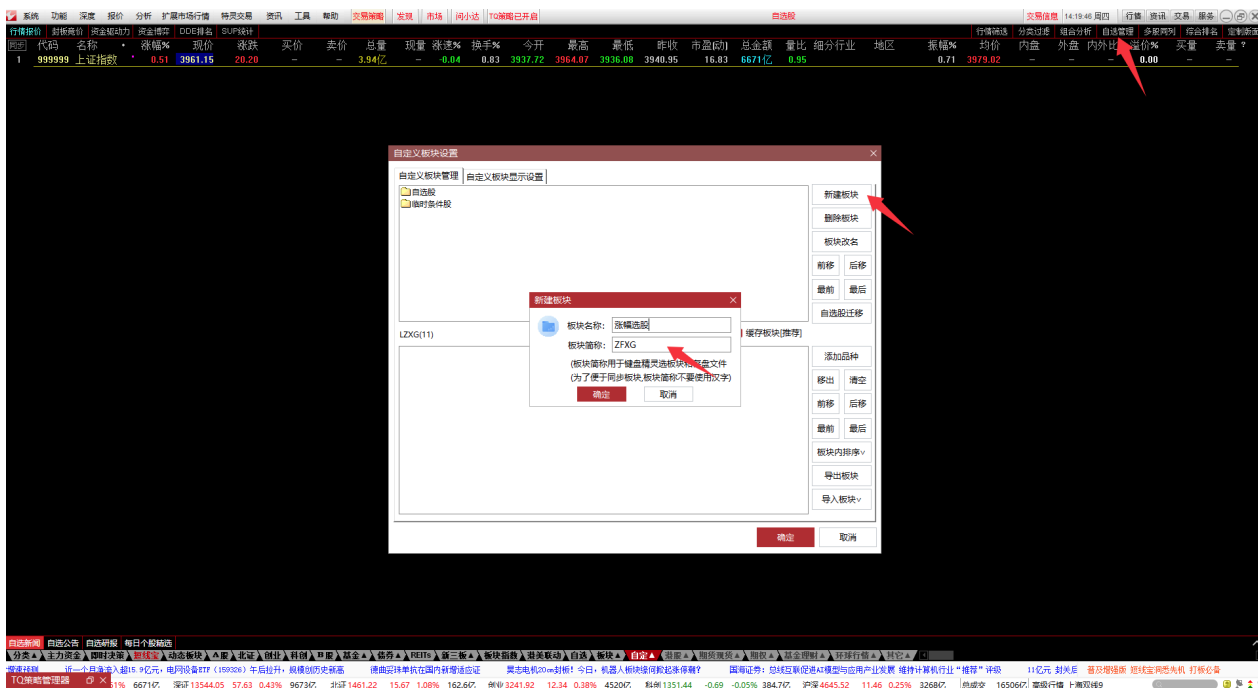
← 安装Python及开发环境

快速开始第一个策略 →

步骤分解

一个完整选股入自定义板块策略只需要两步:

第一步：客户端新增自定义板块



第二步：在VSCode里面运行以下python代码

实现运行函数：在这个策略里, 我们会根据运行结果做出相应操作:

```
1 # 策略说明：如果运行时间点价格高出昨收5%，则进入涨幅选股板块，否则清空该板块
2 import pandas as pd
3 import numpy as np
4 from datetime import datetime
5 from tqcenter import tq
6
7 # 初始化tq
8 tq.initialize(__file__)
9
10 # 1. 基础配置
```

py



```
13 target_end = datetime.now().strftime('%Y%m%d') # 数据结束日期 (当日)
14 target_gain = 5.0 # 目标涨幅 (%), 可
15 target_block_name = 'ZFXG' # 目标自定义板块简称
16
17 # 2. 获取并整理收盘价数据
18 df_real = tq.get_market_data(
19     field_list=['Close'],
20     stock_list=batch_codes,
21     start_time=start_time,
22     end_time=target_end,
23     dividend_type='front', # 前复权
24     period='1d', # 日线
25     fill_data=True # 填充缺失数据
26 )
27 # 转换为「日期×股票代码」的收盘价宽表
28 close_df = tq.price_df(df_real, 'Close', column_names=batch_codes)
29
30 # 3. 核心: 计算当日相较于昨日的涨幅 (%)
31 # 昨日收盘价 (向下平移1行)
32 prev_close = close_df.shift(1)
33 # 计算涨幅: (当日收盘价 - 昨日收盘价) / 昨日收盘价 × 100%
34 daily_gain = (close_df - prev_close) / prev_close * 100
35
36 # 4. 筛选符合条件的股票 (最新交易日涨幅超target_gain%)
37 latest_date = daily_gain.index[-1] # 最新交易日
38 latest_daily_gain = daily_gain.loc[latest_date] # 每只股票最新交易日的涨幅
39 # 筛选条件: 涨幅 > target_gain% (排除NaN, 避免数据异常)
40 target_stocks = latest_daily_gain[latest_daily_gain > target_gain].sort_val
41 target_stocks_list = target_stocks.index.tolist() # 提取符合条件的股票代码列
42
43 # 5. 结果输出与自定义板块操作 (可按需注释)
44 print(f"\n=== 筛选结果 (当日涨幅>{target_gain}%) ===")
45 if not target_stocks.empty:
46     # ===== 模块1: 打印筛选结果 =====
47     print("【模块1: 打印筛选结果】")
48     print(f"符合条件的股票共 {len(target_stocks)} 只: ")
49     print(f"{'股票代码':<12} {'昨日收盘价':<12} {'当日收盘价':<12} {'当日涨幅'
50     print("-" * 50)
51     for stock_code, gain in target_stocks.items():
52         prev_price = prev_close.loc[latest_date, stock_code]
53         curr_price = close_df.loc[latest_date, stock_code]
54         print(f"{'stock_code':<12} {'prev_price':<12.2f} {'curr_price':<12.2f} {'g
55     print("-" * 50)
```



```
58     try:
59         print("【模块2: 自定义板块操作】")
60         tq.send_user_block(block_code=target_block_name, stocks=target_stocks)
61         print(f"✅ 已成功将股票添加至自定义板块「{target_block_name}」")
62     except Exception as e:
63         print(f"❌ 添加自定义板块失败: {e}")
64     print("-" * 50)
65
66
67
68     else:
69         # ===== 模块1: 打印空结果 =====
70         print("【模块1: 打印筛选结果】")
71         print(f"暂无当日涨幅 > {target_gain}%的股票")
72         print("-" * 50)
73
74         # ===== 模块2: 清空自定义选板块 =====
75         try:
76             print("【模块2: 自定义板块操作】")
77             tq.send_user_block(block_code=target_block_name, stocks=[], show=True)
78             print(f"✅ 已清空自定义板块「{target_block_name}」")
79         except Exception as e:
80             print(f"❌ 清空自定义板块失败: {e}")
81         print("-" * 50)
82
83
```

结果示例

VSCode端



TDX数据接口自动初始化成功，使用路径：d:\new_tdx_test\PYPlugins\user\涨幅选股.py

=== 筛选结果（当日涨幅>5.0%）===

【模块1：打印筛选结果】

符合条件的股票共 4 只：

股票代码	昨日收盘价	当日收盘价	当日涨幅
300274.SZ	167.88	183.58	9.35 %
300699.SZ	32.16	35.01	8.86 %
603993.SH	18.70	19.94	6.63 %
601899.SH	32.03	33.69	5.18 %

【模块2：自选板块操作】

✓ 已成功将股票添加至自选板块「ZFXG」

TDX数据连接已自动关闭

PS D:\new_tdx_test\PYPlugins\user> |

通达信终端

代码	名称	涨幅%	现价	涨跌	买价	卖价	总量	现量	涨幅%	换手%	今开	最高	最低	昨收	市盈率	总市值	量比	细分行业	地区	振幅%	均价	内盘	外盘	内外比	溢价%	买量	卖量
1	300274 阳光电源	9.66	184.09	16.21	184.09	184.12	820288	230	0.82	5.16	169.88	184.12	168.85	167.88	24.09	145.7亿	3.98	光伏逆变器	安徽	9.10	177.56	303389	516899	0.59	0.00	2	142
2	300699 光威复材	8.86	35.01	2.85	35.00	35.01	749792	244	-0.13	9.13	33.33	36.66	33.13	32.16	52.62	26.11亿	7.07	碳纤维	山东	10.98	34.78	291898	457893	0.64	0.00	7440	338
3	603993 洛阳钼业	6.50	19.93	1.23	19.93	19.94	158.9万	132	-0.19	0.91	19.04	19.99	18.98	18.70	22.39	30.9亿	2.07	钨钼	河南	5.40	19.47	548906	104.0万	0.53	0.00	5	95
4	601899 紫金矿业	5.12	33.67	1.64	33.66	33.68	165.6万	436	0.03	0.80	32.60	33.77	32.55	32.03	17.73	54.8亿	2.01	铜	福建	3.81	33.10	591167	106.5万	0.56	0.00	226	19

← 安装通达信终端并获取数据

初始化initialize →



很抱歉，您访问的网页出现内部错误，请稍后再试！

[返回首页](#)

订阅行情subscribe_hq

订阅股票实时更新

```
1 subscribe_hq(stock_list: List[str] = [], callback = None):
```

py

输入参数

参数	是否必选	参数类型	参数说明
stock_list	Y	List[str]	订阅的证券代码
callback	Y	str	回调函数

- 订阅股票更新 传入回调函数，订阅的股票有更新时，系统会调用回调函数，最多订阅100条
- 回调函数格式定义为on_data(datas) datas格式为 {"Code": "XXXXXX.XX", "ErrorId": "0"}

接口使用

```
1 from tqcenter import tq
2
3 tq.initialize(__file__)
4
5 # 回调函数 功能为收到更新后请求最新的report数据
6 def my_callback_func(data_str):
7     print("Callback received data:", data_str)
8     code_json = json.loads(data_str)
9     print(f"codes = {code_json.get('Code')}")
10    report_ptr = tq.get_report_data(code_json.get('Code'))
11    print(report_ptr)
12    return None
13
14 sub_hq = tq.subscribe_hq(stock_list=['688318.SH'], callback=my_callback_func)
15 print(sub_hq)
```

py



```
18 while True:
19     #     time.sleep(1)
20
```

数据样本

```
1  {
2      "Error" : "订阅688318.SH更新成功.",
3      "ErrorId" : "0",
4      "run_id" : "1"
5  }
```

← [初始化initialize](#)

[取消订阅更新unsubscribe_hq](#) →

取消订阅更新unsubscribe_hq

取消订阅股票实时更新

```
1 unsubscribe_hq(stock_list: List[str] = []):
```

py

输入参数

参数	是否必选	参数类型	参数说明
stock_list	Y	List[str]	证券代码

- 订阅股票更新 传入回调函数，订阅的股票有更新时，系统会调用回调函数，最多订阅100条
- 回调函数格式定义为on_data(datas) datas格式为 {"Code":"XXXXXX.XX","ErrorId":"0"}

接口使用

```
1 from tqcenter import tq
2 tq.initialize(__file__)
3 un_sub_ptr = tq.unsubscribe_hq(stock_list=['688318.SH'])
4 print(un_sub_ptr)
```

py

数据样本

```
1 {
2     "Error" : "取消全部订阅更新失败.",
3     "ErrorId" : "0",
4     "run_id" : "1"
5 }
```



← [订阅行情subscribe_hq](#)

[获得订阅列表get_subscribe_hq_stock_list](#) →

获得订阅列表

get_subscribe_hq_stock_list

获得当前策略订阅的股票列表

```
1 get_subscribe_hq_stock_list():
```

py

接口使用

```
1 from tqcenter import tq
2
3 tq.initialize(__file__)
4
5 sub_list = tq.get_subscribe_hq_stock_list()
6 print(sub_list)
```

py

数据样本

```
1 ['600519.SH']
```

← [取消订阅更新unsubscribe_hq](#)

[刷新行情缓存refresh_cache](#) →

刷新行情缓存(最新report和K线数据)refresh_cache

刷新行情缓存(最新snapshot和K线数据)。如果不调用，则在首次取snapshot和K线时系统会自动刷新一次行情

```
1 refresh_cache()
```

py

接口使用

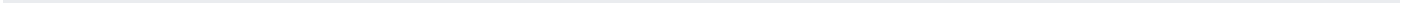
```
1 from tqcenter import tq
2
3 tq.initialize(__file__)
4 refresh_cache = tq.refresh_cache()
5 print(refresh_cache)
```

py

数据样本

使用后会客户端弹出刷新数据的加载界面，加载完成后才会有返回

```
1 {
2     "Error" : "Refresh Cache Success.",
3     "ErrorId" : "0",
4     "run_id" : "1"
5 }
```



刷新历史K线缓存refresh_kline

根据股票和周期刷新历史K线缓存，如果本地没有下载完整的日线等数据，则可以调用这个函数定向下载某些品种某些周期的历史K线数据

```
1 refresh_kline(stock_list: List[str] = [], period: str = '')
```

输入参数

参数	是否必选	参数类型	参数说明
stock_list	Y	List[str]	证券代码列表，证券代码格式为6位数+市场后缀(.SH/.SZ/.BJ等)
period	Y	str	周期 1d为日线、1m为一分钟线、5m为五分钟线，只支持这三种，其它周期的数据均由这三种数据生成

接口使用

```
1 from tqcenter import tq
2 tq.initialize(__file__)
3 refresh_kline = tq.refresh_kline(stock_list=['688318.SH'],period='1d')
4 print(refresh_kline)
```

数据样本

使用后会客户端弹出刷新数据的加载界面，加载完成后才会有返回



```
3     "ErrorId" : "0",  
4     "run_id" : "1"  
5 }
```

← [刷新行情缓存refresh_cache](#)

[发送消息到TQ策略界面send_message](#) →

发送消息到通达信客户端send_message

发送消息给通达信客户端的TQ策略界面

```
1 send_message(msg_str: str) -> Dict:
```

py

输入参数

参数	是否必选	参数类型	参数说明
msg_str	Y	str	消息字符串

- 传入的字符串使用 | 可以让客户端将其分为两条（插入 \n 也可以分行显示）

接口使用

```
1 from tqcenter import tq
2 tq.initialize(__file__)
3 msg_str = "这是第一行。 | 这是第二行。"
4 tq.send_message(msg_str)
```

py

发送预警信号send_warn

往客户端发送指定股票的预警信号

```
1 send_warn(stock_list:      List[str] = [],
2            time_list:      List[str] = [],
3            price_list:     List[str] = [],
4            close_list:     List[str] = [],
5            volum_list:     List[str] = [],
6            bs_flag_list:   List[str] = [],
7            warn_type_list: List[str] = [],
8            reason_list:    List[str] = [],
9            count:          int = 1) -> Dict:
```

py

输入参数

参数	是否必选	参数类型	参数说明
stock_list	Y	List[str]	证券代码列表
time_list	Y	List[str]	时间列表
price_list	N	List[str]	现价列表
close_list	N	List[str]	收盘价列表
volum_list	N	List[str]	成交额列表
bs_flag_list	N	List[str]	买卖标志：0买1卖2未知
warn_type_list	N	List[str]	预警类型：0常规预警（目前仅支持）
reason_list	N	List[str]	预警原因
count	N	int	有效数据个数



- bs_flag_list 0买1卖2未知，长度小于count的会自动补为2。
- reason_list每个元素有效长度为25个汉字（50个英文）|
- count限定入参中每个list中的有效数据个数，即每个list前count个数据会传给客户端
- stock_list与其他list的元素数据是一一对应的，即stock_list的第一个元素对应的预警信息是其他list的第一个元素，同一只股票的多个预警信息，则在stock_list中加入多次该股票

接口使用

```
1 from tqcenter import tq
2 tq.initialize(__file__)
3 warn_res = tq.send_warn(stock_list = ['688318.SH', '688318.SH', '600519.SH'],
4                             time_list = ['20251215141115', '20251215142100', '20251215143100'],
5                             price_list= ['123.45', '133.45', '1823.45'],
6                             close_list= ['122.50', '132.50', '1822.50'],
7                             volum_list= ['1000', '2000', '15000'],
8                             bs_flag_list= ['0'],
9                             warn_type_list= ['0'],
10                            reason_list= ['价格突破预警线', '收盘价突破预警线', '成交量突破预警线'],
11                            count=3)
12 print(warn_res)
```

数据样本

```
1 {'Error': '发送预警信号成功.', 'ErrorId': '0', 'run_id': '1'}
```

← [发送消息到TQ策略界面send_message](#)

[发送文件到客户端send_file](#) →

发送文件到客户端send_file

往通达信客户端发送文件名，可由TQ策略数据浏览中打开

```
1 send_file(file: str) -> Dict:
```

py

输入参数

参数	是否必选	参数类型	参数说明
file	Y	str	文件路径

- 文件放于 .\PYPlugins\file\ 文件夹中时，file可仅传入文件名
- 文件放于其他位置时，file需要传入绝对路径
- 目前支持的文件类型：txt, pdf, html

接口使用

```
1 from tqcenter import tq
2 tq.initialize(__file__)
3 file = "test.txt"
4 tq.send_file(file)
```

py

发送回测数据send_bt_data

往客户端发送指定股票的回测数据

```
1 send_bt_data(stock_code:      str = '',
2               time_list:      List[str] = [],
3               data_list:      List[List[str]] = [],
4               count:          int = 1) -> Dict:
```

输入参数

参数	是否必选	参数类型	参数
stock_code	Y	List[str]	证券代
time_list	Y	List[str]	时间列
data_list	N	List[List[str]]	回测数
count	N	int	有效数

- data_list为二维List，每个子元素对应time_list的一个元素时间点，且每个子元素最多有16个有效纯数字字符串，即data_list每个子List的前16个数据为一个时间点的有效数据
- count限定入参中每个list中的有效数据个数，即每个list前count个数据会传给客户端

接口使用

```
1 from tqcenter import tq
2
3 tq.initialize(__file__)
4
5 bt_data = tq.send_bt_data(stock_code = '688318.SH',
6                           time_list = ['20251215141115'],
7                           data_list = [['11']],
8                           count = 1)
```



数据样本

1

```
{'Error': '发送回测结果成功.', 'ErrorId': '0', 'run_id': '1'}
```

← [发送文件到客户端send_file](#)

[下载特定数据文件download_file](#) →

下载特定数据文件（10大股东或ETF申赎数据） download_file

下载10大股东数据或ETF申赎数据

```
1 download_file(stock_code: str = '',
2               down_time: str = '',
3               down_type: int = 1):
```

py

输入参数

参数	是否必选	参数类型	参数说明
stock_code	Y	List[str]	证券代码
down_time	Y	List[str]	指定时间
down_type	Y	List[str]	指定下载类型

- down_type=1时，下载10大股东数据，down_time只生效年份
- down_type=2时，下载ETF申赎清单，down_time生效到日期
- 下载的文件保存在 .\PYPlugins\data 文件夹

接口使用

```
1 from tqcenter import tq
2 tq.initialize(__file__)
3 # 下载10大股东数据
4 down_ptr_10 = tq.download_file(stock_code='688318.SH', down_time='20250101')
5 print(down_ptr_10)
6 # 下载ETF申赎数据
7 down_ptr_etf = tq.download_file(stock_code='159109.SH', down_time='20250101')
8 print(down_ptr_etf)
```

py

数据样本

```
1  {
2    "ErrorId" : "0",
3    "Msg" : "下载十大股东数据[2025]成功。",
4    "run_id" : "1"
5  }
6
7  {
8    "ErrorId" : "0",
9    "Msg" : "下载ETF申述清单[20250101]成功。",
10   "run_id" : "1"
11  }
```

← [发送回测数据send_bt_data](#)

[获取交易日列表get_trading_dates](#) →

获取交易日列表get_trading_dates

根据指定时间段获取交易日列表

```
1 get_trading_dates(market: str,
2                   start_time: str,
3                   end_time: str,
4                   count:int = -1) -> List:
```

py

输入参数

参数	是否必选	参数类型	参数说明
market	Y	str	市场代码（暂固定为SH）
start_time	N	str	起始日期
end_time	N	str	结束日期
count	N	int	返回最近的count个交易日

- 需要现在客户端下载上证指数（999999）的盘后数据 目前仅支持A股
- count > 0时，限制返回从结束日期往前最近的count个在限定时间段中的交易日

接口使用

```
1 from tqcenter import tq
2
3 tq.initialize(__file__)
4
5 trade_dates = tq.get_trading_dates(market = 'SH', start_time = '20220101',
6 print(trade_dates)
```

py

数据样本



← [下载特定数据文件download_file](#)

[获取K线数据get_market_data](#) →



很抱歉，您访问的网页出现内部错误，请稍后再试！

[返回首页](#)

获取快照数据get_market_snapshot

根据股票，获取最新行情数据

```
1 get_market_snapshot(stock_code: str) -> Dict:
```

py

输入参数

参数	是否必选	参数类型	参数说明
stock_code	Y	str	证券代码

返回参数

参数	默认返回	参数类型	参数说明
ItemNum	Y	str	采样点数
Open	Y	str	开盘价
High	Y	str	最高价
Low	Y	str	最低价
LastClose	Y	str	前收盘价
Now	Y	str	现价
RefreshNum	Y	str	刷新数/行情时间
Volume	Y	str	总手
NowVol	Y	str	现手
Amount	Y	str	总成交金额
Inside	Y	str	内盘



Outside	Y	str	外盘
TickDiff	Y	str	笔涨跌
InOutFlag	Y	str	内外盘标志： 0:Buy 1:Sell 2:None
CJBS	Y	str	成交笔数
Jjz	Y	str	基金净值
Buyp	Y	List[str]	五个买价
Buyv	Y	List[str]	对应的五个买盘量
Sellp	Y	List[str]	五个卖价
Sellv	Y	List[str]	对应的五个卖盘量

接口使用

获取688318.SH从2025-12-20到今为止最新一条日K线的不复权数据

```
1 from tqcenter import tq
2
3 tq.initialize(__file__)
4
5 market_snapshot = tq.get_market_snapshot(stock_code = '688318.SH')
6 print(market_snapshot)
```

py

数据样本

```
1 {'Amount': '33832.39',
2  'Buyp': ['131.32', '0.00', '0.00', '0.00', '0.00'],
3  'Buyv': ['3', '0', '0', '0', '0'],
4  'CJBS': '0',
5  'ErrorId': '0',
6  'InOutFlag': '0',
7  'Inside': '10060',
8  'ItemNum': '2079',
```



```
11      'Max': '131.87',  
12      'Min': '128.00',  
13      'Now': '131.29',  
14      'NowVol': '25',  
15      'Open': '128.01',  
16      'Outside': '15889',  
17      'RefreshNum': '0',  
18      'Sellp': ['131.33', '0.00', '0.00', '0.00', '0.00'],  
19      'Sellv': ['10', '0', '0', '0', '0'],  
20      'TickDiff': '0.11',  
21      'Volume': '25949'}
```

← [获取K线数据get_market_data](#)

[获取证券基本信息get_stock_info](#) →

获取证券基本信息get_stock_info

根据股票，获取股票基础的财务数据

```
1 get_stock_info(cls,  
2     stock_code:str,  
3     field_list: List = []) -> Dict:
```

py

输入参数

参数	是否必选	参数类型	参数说明
field_list	Y	List[str]	字段筛选，不能为空
stock_code	Y	str	证券代码

返回参数

参数	默认返回	参数类型	参数说明
Name	Y	str	证券名称
Unit	Y	str	交易单位
VolBase	Y	str	量比的基量
Fz[8]	Y	List[str]	开收市时间（4段）
InitTimer	Y	str	初始化时间
EndTimer	Y	str	收盘时间
DelayMin	Y	str	延时分钟数
BelongHS300	Y	str	是否属于沪深300板块
BelongHaskQZ	Y	str	是否属于可转债板块



BelongRZRQ	Y	str	是否属于融资融券板块
IsQH	Y	str	是否属于期货品种
IsHKGP	Y	str	是否是港股品种
Vol_BaseRate	Y	str	期货的每手乘数 期权合约单位
MinPrice	Y	str	最小变动价位
IsQQ	Y	str	是否是期权品种
ActiveCapital	Y	str	流通股本
J_start	Y	str	上市日期
J_addr	Y	str	所属省份
J_hy	Y	str	所属行业
J_zgb	Y	str	总股本
TodayDRFlay	Y	str	今日的除权除息标识 1:分红 2:送转股 3:分红+送转股 4:送股
HSStockKind	Y	str	沪深品种类型
BelongHSGT	Y	str	是否属于陆股通板块
IsZS	Y	str	是否是指数
J_bg	Y	str	B股
J_hg	Y	str	H股
J_zzc	Y	str	总资产
J_ldzc	Y	str	流动资产
J_gdzc	Y	str	固定资产
J_wxzc	Y	str	无形资产
J_gdrs	Y	str	股东人数



J_ldtz	Y	str	流动负债
J_cqfz	Y	str	少数股东权益
J_zbgjj	Y	str	资本公积金
J_jzc	Y	str	股东权益（净资产）
J_yysy	Y	str	营业收入
J_yycb	Y	str	营业成本
J_yszk	Y	str	应收账款
J_yyly	Y	str	营业利润
J_tzsy	Y	str	投资收益
J_jyxjl	Y	str	经营现金净流量
J_zxjl	Y	str	总现金净流量
J_ch	Y	str	存货
J_lyze	Y	str	利润总额
J_shly	Y	str	税后利润
J_jly	Y	str	净利益
J_wfpjy	Y	str	未分配利益
J_jyl	Y	str	净益率
J_mgwfp	Y	str	每股未分配
J_mgsy	Y	str	每股收益（折算为全年）
J_mgsy2	Y	str	季报每股收益 (财报中提供的每股收益)
J_mggjj	Y	str	每股公积金
J_mgjzc	Y	str	每股净资产



J_mgjzc2	Y	str	季报每股净资产 (财报中提供的每股收益)
J_gdqyb	Y	str	股东权益比

接口使用

```
1 from tqcenter import tq
2 tq.initialize(__file__)
3 fdc = tq.get_stock_info(stock_code='688318.SH', field_list=[])
4 print(fdc)
```

py

数据样本

```
1 {'ActiveCapital': '25611.94',
2  'BelongHS300': '0',
3  'BelongHSGT': '1',
4  'BelongHaskQZ': '0',
5  'BelongRZRQ': '1',
6  'DelayMin': '0',
7  'EndTimer': '0',
8  'ErrorId': '0',
9  'Fz': ['570', '690', '780', '900', '900', '900', '900', '900'],
10 'HSStockKind': '4',
11 'InitTimer': '0',
12 'IsHKGP': '0',
13 'IsQH': '0',
14 'IsQQ': '0',
15 'IsZS': '0',
16 'J_addr': '18',
17 'J_bg': '0.00',
18 'J_ch': '61.84',
19 'J_cqfz': '73.15',
20 'J_gdqyb': '-1.08',
21 'J_gdrs': '24154.00',
22 'J_gdzc': '972.62',
23 'J_hg': '0.00',
24 'J_hy': '24',
25 'J_jly': '18421.34',
```



```
28 'J_jzc': '370434.03',
29 'J_ldfz': '17412.97',
30 'J_ldzc': '235598.84',
31 'J_lyze': '20829.85',
32 'J_mggjj': '6.17',
33 'J_mgjzc': '14.46',
34 'J_mgjzc2': '14.46',
35 'J_mgsy': '0.96',
36 'J_mgsy2': '0.00',
37 'J_mgwfp': '6.85',
38 'J_shly': '18421.45',
39 'J_start': '20200427',
40 'J_tzsy': '5091.96',
41 'J_wfply': '175521.63',
42 'J_wxzc': '1184.64',
43 'J_yszk': '2726.99',
44 'J_yycb': '4258.70',
45 'J_yyly': '20836.07',
46 'J_yysy': '19827.85',
47 'J_zbgjj': '157998.02',
48 'J_zgb': '25611.94',
49 'J_zxjl': '9779.30',
50 'J_zzc': '389036.97',
51 'MinPrice': '0.01',
52 'Name': '财富趋势',
53 'TodayDRFlag': '0',
54 'Unit': '100',
55 'VolBase': '118.48',
56 'VolBaseRate': '0'}
```

← [获取快照数据get_market_snapshot](#)

[获取分红配送数据get_divid_factors](#) →

获取分红配送数据get_divid_factors

根据股票，获取指定时间段内的分红配送数据

```
1 get_divid_factors(stock_code: str,  
2                 start_time: str,  
3                 end_time: str) -> pd.DataFrame:
```

py

输入参数

参数	是否必选	参数类型	参数说明
stock_code	Y	str	证券代码
start_time	N	str	起始时间
end_time	N	str	结束时间

返回参数

参数	默认返回	参数类型	参数说明
Type	Y	str	---
Bonus	Y	str	---
AlloPrice	Y	str	---
ShareBonus	Y	str	---
Allotment	Y	str	---

接口使用

获取688318.SH全部分红配送数据



```
3 divid_factors = tq.get_divid_factors(  
4     stock_code='688318.SH',  
5     start_time='',  
6     end_time='')  
7 print(divid_factors)
```

数据样本

	Date	Type	Bonus	AllotPrice	ShareBonus	Allotment
1						
2						
3	2020-09-29	1	6.0	0.0	0.0	0.0
4	2021-05-27	1	10.0	0.0	0.0	0.0
5	2022-06-20	1	14.0	0.0	4.0	0.0
6	2023-06-13	1	5.0	0.0	4.0	0.0
7	2024-06-14	1	8.0	0.0	4.0	0.0

← [获取证券基本信息get_stock_info](#)

[获取新股申购信息get_ipo_info](#) →

获取新股申购信息get_ipo_info

获取今天及未来的新股或新发债申购信息

```
1 get_ipo_info(ipo_type:int = 0,  
2             ipo_date:int = 0):
```

py

输入参数

参数	是否必选	参数类型	参数说明
ipo_type	Y	str	自定义板块简称
ipo_date	Y	int	自定义板块名称

- ipo_type=0 表示获取新股申购信息
- ipo_type=1 表示获取新发债信息
- ipo_type=2 表示获取新股和新发债信息
- ipo_date=0 表示只获取今天信息
- ipo_date=1 表示获取今天及以后信息

接口使用

```
1 from tqcenter import tq  
2 tq.initialize(__file__)  
3 ipo_info = tq.get_ipo_info(ipo_type=2, ipo_date=1)  
4 print(ipo_info)
```

py

数据样本



← [获取分红配送数据get_divid_factors](#)

[获取专业财务数据get_financial_data](#) →



很抱歉，您访问的网页出现内部错误，请稍后再试！

[返回首页](#)

获取指定日期专业财务数据

get_financial_data_by_date

根据股票，获取指定日期的专业财务数据，与基础财务数据不同，需要先在客户端种下载专业财务数据

```
1 get_financial_data_by_date(stock_list: List[str] = [],  
2                             field_list: List[str] = [],  
3                             year: int = 0,  
4                             mmdd: int = 0) -> Dict:
```

py

输入参数

参数	是否必选	参数类型	参数说明
field_list	Y	List[str]	字段筛选，不能为空（如 FN193 ）
stock_list	Y	List[str]	证券代码列表
year	N	str	指定年份
mmdd	N	str	指定月日

输出参数

同get_financial_data一样。

接口使用

```
1 from tqcenter import tq  
2  
3 tq.initialize(__file__)  
4
```

py



```
7         field_list = ['FN193', 'FN194', 'FN195', 'FN196', 'FN197'],
8         year=0,
9         mdd=0)
10     print(fd)
```

数据样本

```
1     {'600519.SH':
2     {'FN193': '162.47',
3     'FN194': '69.67',
4     'FN195': '16.07',
5     'FN196': '8.71',
6     'FN197': '25.14'}}}
```

← [获取专业财务数据get_financial_data](#)

[获取股票交易数据get_gpjy_value](#) →

获取股票交易数据get_gpjy_value

根据股票，获取指定时间段内的股票交易数据，需要先客户端种下载股票数据包

```
1 get_gpjy_value(stock_list: List[str] = [],  
2               field_list: List[str] = [],  
3               start_time: str = '',  
4               end_time: str = '') -> Dict:
```

py

输入参数

参数	是否必选	参数类型	参数说明
field_list	Y	List[str]	字段筛选，不能为空
stock_list	Y	List[str]	证券代码列表
start_time	N	str	起始时间
end_time	N	str	结束时间

输出参数

名称	类型	数值	说明
GP01	double		股东人数 股东户数(户)
GP02	double		龙虎榜 买入总计(万元) 卖出总计(万元)[注：该指标展示20230717日之后的数据]
GP03	double		融资融券1 融资余额(万元) 融券余量(股)
GP04	double		大宗交易 成交均价(元) 成交额(万元)
GP05	double		增减持1 成交均价(元) 变动股数(股)



GP06	double		陆股通持股量 持股数量(股)[注：该指标展示20170317日之后的数据]
GP07	double		陆股通市场成交净额 陆股通市场净买入(万元)[注：官方只公布了每日的前十名数据]
GP08	double		龙虎榜机构(卖方)数据 卖方机构个数 机构卖出金额(万元)
GP09	double		龙虎榜机构(买方)数据 买方机构个数 机构买入金额(万元)
GP10	double		近3月机构调研情况 近3月机构调研次数 近3月调研机构数量
GP11	double		融资融券2 融资买入额(万元) 融资偿还额(万元)
GP12	double		融资融券3 融券卖出量(股) 融券偿还量(股)
GP13	double		融资融券4 融资净买入(万元) 融券净卖出(股)
GP14	double		涨停数据 涨停金额(即板上成交,万元) 开板次数[注：该指标展示20180319日之后的数据]
GP15	double		涨跌停 涨跌停状态 封单金额(万元)[注：涨停取2,曾涨停取1,跌停取-2,曾跌停取-1;跌停和曾跌停时,封单金额取负值 该指标展示20160926日之后的数据]
GP16	double		总市值 总市值(万元)
GP17	double		龙虎榜营业部数据 买入金额(万元) 卖出金额(万元)
GP18	double		龙虎榜沪深股通数据 买入金额(万元) 卖出金额(万元)
GP19	double		每周股票质押数量 无限售股份质押数(万) 有限售股份质押数(万)[注：该指标展示20180316日之后的数据]
GP20	double		每周股票质押比例 质押比例(%) [注：该指标展示20180316日之后的数据]
GP21	double		股息率 股息率(%)
GP22	double		涨跌停 封成比 封流比[注：该指标展示20180319日之后的数据]
GP23	double		拟增减持 拟增持数量(万股) 拟减持数量(万股)

GP24	double		涨停 首次涨停时间 涨停最大封单额(万) [注：首次涨停时间展示20160301之后的数据，涨停最大封单额展示20200730之后的数据]
GP25	double		盘前盘后成交量 开盘成交量(手) 盘后固定成交量(手) [注：盘后固定成交量只包含科创板和创业板]
GP26	double		拟增减持金额 拟增持金额(万元) 拟减持金额(万元)
GP27	double		人气排名 市场人气排名 行业人气排名 [注：行业排名为通达信二级研究行业排名]
GP28	double		股票回购 回购均价(元) 回购数量(万股)
GP29	double		证券信息 是否复牌日 是否更名日 [注：是否复牌日说明：0-不是复牌日，n(n>0)-停牌n个交易日之后的复牌日；是否更名日说明：0-未更名，1-常规更名，2-加ST，3-加*ST，4-摘帽，5-其他]
GP30	double		分红送转 派息金额(万元) 送转数量(股) [注：对应展示日期为除权除息日]
GP31	double		转融券 期初余量(股) 期末余量(股)
GP32	double		转融券 融出数量(股) 融出市值(元)
GP33	double		跌停数据 跌停金额(万元) 开板次数 [注：该指标展示20180319日之后的数据,暂无跌停金额数据]
GP34	double		跌停 首次跌停时间 跌停最大封单额(万) [注：首次跌停时间展示20160301之后的数据，跌停最大封单额展示20200730之后的数据]
GP35	double		增减持2 增持数量(股) 减持数量(股)
GP36	double		竞价涨停买 买入金额(万元) [注：该指标展示20241101日之后的数据]
GP37	double		龙虎榜2 上榜类型连续交易日(天) [注：该指标展示上榜类型中指代的连续交易日类型]
GP38	double		涨停相关1 近1年涨停次数 近1年溢价5%次数
GP39	double		涨停相关2 近1年首板封板率(%) 近1年次日红盘率(%)
GP40	double		涨停相关3 近1年连板率(%) 最后涨停时间



GP41	double		股权登记日 配股股权登记日
GP42	double		龙虎榜专业机构买卖净额 买方成交净额(万元) 卖方成交净额(万元)
GP43	double		配股实施 配股价格(元) 配股数量(万股)
GP44	double		股票评分 综合评分
GP45	double		评级系数 评级系数
GP46	double		拟询价转让 拟转让股数(万股) 拟转让占总股本(%)

接口使用

```
1 from tqcenter import tq
2
3 tq.initialize(__file__)
4
5 gp_val = tq.get_gpjy_value(
6     stock_list=['688318.SH'],
7     field_list=['GP1', 'GP2', 'GP3', 'GP4', 'GP5'],
8     start_time='20250101',
9     end_time='20250102')
10 print(gp_val)
```

py

数据样本

```
1 {'688318.SH': {'GP3': [['141405.89', '11113.00']]}}
```

← [获取指定日期财务数据get_financial_data_by_date](#)

[获取指定日期股票交易数据get_gpjy_value_by_date](#) →

获取指定日期股票交易数据

get_gpjy_value_by_date

根据股票，获取指定时间段内的股票交易数据，需要先客户端种下载股票数据包

```
1 def get_gpjy_value_by_date(stock_list: List[str] = [],  
2                             field_list: List[str] = [],  
3                             year: int = 0,  
4                             mmdd: int = 0) -> Dict:
```

py

输入参数

参数	是否必选	参数类型	参数说明
field_list	Y	List[str]	字段筛选，不能为空
stock_list	Y	List[str]	证券代码列表
year	N	str	指定年份
mmdd	N	str	指定月日

输出参数

同get_gpjy_value一样。

接口使用

```
1 from tqcenter import tq  
2  
3 tq.initialize(__file__)  
4  
5 gp_one = tq.get_gpjy_value_by_date(
```

py



```
8         year = int(input())
9         print(gp_one)
```

数据样本

```
1         {'688318.SH': {'GP1': ['24154.00', '0.00'], 'GP2': ['20574.12', '18728.85']}}
```

← [获取股票交易数据get_gpjy_value](#)

[获取板块交易数据get_bkgy_value](#) →

获取板块交易数据get_bkjy_value

根据板块代码，获取指定时间段内的板块交易数据，需要先在客户端种下载股票数据包

```
1 get_bkjy_value(stock_list: List[str] = [],  
2               field_list: List[str] = [],  
3               start_time: str = '',  
4               end_time: str = '') -> Dict:
```

py

输入参数

参数	是否必选	参数类型	参数说明
field_list	Y	List[str]	字段筛选，不能为空
stock_list	Y	List[str]	证券代码列表
start_time	N	str	起始时间
end_time	N	str	结束时间

输出参数

名称	类型	数值	说明
BK5	double		市盈率TTM 整体法 算术平均
BK6	double		市净率MRQ 整体法 算术平均
BK7	double		市销率TTM 整体法 算术平均
BK8	double		市现率TTM 整体法 算术平均
BK9	double		涨跌数 上涨家数 下跌家数
BK10	double		板块总市值(亿元) 整体法 算术平均



BK11	double		板块流通市值(亿元) 整体法 算术平均
BK12	double		涨停数 涨停家数 曾涨停家数[注：该指标展示20160926日之后的数据]
BK13	double		跌停数 跌停家数 曾跌停家数[注：该指标展示20160926日之后的数据]
BK14	double		涨停数据 市场高度(不含ST股和未开板新股) 2板及以上涨停个数(不含ST股和未开板新股)[注：该指标展示20180319日之后的数据]
BK15	double		融资融券 沪深京融资余额(万元) 沪深京融券余额(万元)
BK16	double		陆股通资金流入 沪股通流入金额(亿元) 深股通流入金额(亿元) [注：该指标展示20170320日之后的数据]
BK17	double		开盘成交数 开盘成交额(万元) 开盘成交量(万股)
BK18	double		板块股息率(%) 算术平均 整体法
BK19	double		板块自由流通市值(亿元) 整体法 算术平均

接口使用

```
1 from tqcenter import tq
2
3 tq.initialize(__file__)
4
5 bk_data = tq.get_bkgy_value(stock_list=['880660.SH'],
6                               field_list=['BK5', 'BK6', 'BK7', 'BK8', 'BK9'],
7                               start_time='20250101',
8                               end_time='20250102')
9 print(bk_data)
```

数据样本



← [获取指定日期股票交易数据get_gpjy_value_by_date](#)

[获取指定日期板块交易数据get_bkgy_value_by_date](#) →

获取指定日期板块交易数据

get_bkgy_value_by_date

根据板块代码，获取指定日期的板块交易数据，需要先客户端种下载股票数据包

```
1 get_bkgy_value_by_date(stock_list: List[str] = [],  
2                         field_list: List[str] = [],  
3                         year: int = 0,  
4                         mmdd: int = 0) -> Dict:
```

py

输入参数

参数	是否必选	参数类型	参数说明
field_list	Y	List[str]	字段筛选，不能为空
stock_list	Y	List[str]	证券代码列表
year	N	str	指定年份
mmdd	N	str	指定月日

输出参数

同get_bkgy_value一样。

接口使用

```
1 from tqcenter import tq  
2  
3 tq.initialize(__file__)  
4  
5 bk_one = tq.get_bkgy_value_by_date(stock_list=['880660.SH'],
```

py



```
8 print(bk_one)
```

数据样本

```
1
```

```
{'880660.SH': {'BK10': ['6705.83', '191.60'], 'BK11': ['6183.65', '176.68']}}
```

← [获取板块交易数据get_bkgy_value](#)

[获取市场交易数据get_scjy_value](#) →

获取市场交易数据

获取指定时间段内的市场交易数据，需要先在客户端种下载股票数据包

```
1 get_scjy_value(field_list: List[str] = [],
2               start_time: str = '',
3               end_time: str = '') -> Dict: py
```

输入参数

参数	是否必选	参数类型	参数说明
field_list	Y	List[str]	字段筛选，不能为空
start_time	N	str	起始时间
end_time	N	str	结束时间

输出参数

名称	类型	数值	说明
SC01	double		融资融券 沪深京融资余额(万元) 沪深京融券余额(万元)
SC02	double		陆股通资金流入 沪股通流入金额(亿元) 深股通流入金额(亿元)[注：沪股通限制展示2000条数据，深股通展示自20161205以后的数据]
SC03	double		沪深京涨停股个数 涨停股个数 曾涨停股个数 [注：该指标展示20160926日之后的数据]
SC04	double		沪深京跌停股个数 跌停股个数 曾跌停股个数 [注：该指标展示20160926日之后的数据]



SC05	double		上证50股指期货 净持仓(手)[注：该指标展示20171009日之后的数据]
SC06	double		沪深300股指期货 净持仓(手) [注：该指标展示20171009日之后的数据]
SC07	double		中证500股指期货 净持仓(手) [注：该指标展示20171009日之后的数据]
SC08	double		ETF基金规模份额数据 ETF基金规模(亿份) ETF净申赎(亿份)
SC09	double		沪月新开A股账户 沪月新开A股账户(万户)
SC10	double		增减持统计 增持额(万元) 减持额(万元)[注：部分公司公告滞后,造成每天查看的数据可能会不一样]
SC11	double		大宗交易 溢价的大宗交易额(万元) 折价的大宗交易额(万元)
SC12	double		限售解禁 限售解禁计划额(亿元) 限售解禁股份实际上市金额(亿元)[注：该指标展示201802月之后的数据;部分股票的解禁日期延后，造成不同日期提取的某天的计划额可能不同]
SC13	double		分红 市场总分红额(亿元)[注：除权派息日的A股市场总分红额]
SC14	double		募资 市场总募资额(亿元)[注：发行日期/除权日期的首发、配股和增发的总募资额]
SC15	double		打板资金 封板成功资金(亿元) 封板失败资金(亿元) [注：该指标展示20160926日之后的数据]
SC16	double		龙虎榜 买入总金额(亿元) 卖出总金额(亿元)
SC17	double		龙虎榜机构数据 买入金额(亿元) 卖出金额(亿元)
SC18	double		龙虎榜营业部数据 买入金额(亿元) 卖出金额(亿元)
SC19	double		龙虎榜沪深股通数据 买入金额(亿元) 卖出金额(亿元)
SC20	double		陆股通净买入 沪股通净买入额(亿元) 深股通净买入额(亿元)



SC21	double		每周无限售质押率 深市质押率(%) 沪市质押率(%) [注：该指标展示20180128日之后的数据]
SC22	double		每周有限售质押率 深市质押率(%) 沪市质押率(%) [注：该指标展示20180128日之后的数据]
SC23	double		连板家数 连板股个数(包含ST和未开板新股) 连板股个数(不含ST股和未开板新股) [注：该指标展示20180319日之后的数据]
SC24	double		沪深京涨跌停股个数 涨停股个数(不含ST股和未开板新股) 跌停股个数 (不含ST股) [注：该指标展示20160926日之后的数据]
SC25	double		融资融券 沪深京融资买入额 (万元) 沪深京融券卖出量 (万股)
SC26	double		每周市场质押比 每周市场质押比例 (%) [注：该指标展示20180316日之后的数据]
SC27	double		央行公开市场净投放 央行公开市场净投放 (亿元)
SC28	double		历史A股新高新低数 历史新高A股股票个数 历史新低A股股票个数(上市满一年的股票)
SC29	double		120天A股新高新低数 120天新高A股股票个数 120天新低A股股票个数(上市满一年的股票)
SC30	double		涨停数据 市场高度(不含ST股和未开板新股) 2板以上涨停个数(不含ST股和未开板新股)[注：该指标展示20180319日之后的数据]
SC31	double		涨跌家数 涨家数 (剔除停牌) 跌家数 (剔除停牌)
SC32	double		20天A股新高新低数 20天新高A股股票个数 20天新低A股股票个数(上市满一年的股票)
SC33	double		市场总封单金额 涨停封单金额 (亿元) 跌停封单金额 (亿元) [注：该指标展示20160926日之后的数据]
SC34	double		涨跌股成交量 上涨股成交量(万手) 下跌股成交量(万手)
SC35	double		涨停数据 换手板家数 回封率(%) [注：两个指标都剔除了未开板新股，换手板家数展示20190605日之后的数据，回封率展示20180927日之后的数据]



SC36	double		曾涨停个股个数 曾涨停个股数(剔除ST股和未开板新股) 曾跌停个股数(剔除ST股) [注：该指标展示20160926日之后的数据]
SC37	double		转融券 融出市值(亿元) 期末余额(亿元)
SC38	double		ETF基金规模金额数据 ETF基金规模(亿元) ETF净申赎(亿元)
SC39	double		涨跌5%家数 涨幅大于等于5%家数 跌幅大于等于5%家数
SC40	double		陆股通成交 陆股通成交总额(亿元) 陆股通成交总笔(万笔)
SC41	double		中证1000股指期货 净持仓(手) [注：该指标展示20220722日之后的数据]
SC42	double		沪深股通成交金额 沪股通成交总额(亿元) 深股通成交总额(亿元)

接口使用

```
1 from tqcenter import tq
2
3 tq.initialize(__file__)
4
5 sc_val = tq.get_scjy_value(field_list=['SC1','SC2','SC3','SC4','SC5'],
6                               start_time='20250101',end_time='20250102')
7 print(sc_val)
```

数据样本

```
1 {'SC1': [['184712288.00', '999820.06']], 'SC2': [['0.00', '0.00']], 'SC3':
```



获取指定日期市场交易数据[get_scjy_value_by_date](#) →

获取指定日期市场交易数据

get_scjy_value_by_date

获取指定时间的市场交易数据，需要先在客户端种下载股票数据包

```
1 get_scjy_value_by_date(field_list: List[str] = [],  
2                         year: int = 0,  
3                         mmdd: int = 0) -> Dict:
```

py

输入参数

参数	是否必选	参数类型	参数说明
field_list	Y	List[str]	字段筛选，不能为空
year	N	str	指定年份
mmdd	N	str	指定月日

输出参数

同get_scjy_value一样。

接口使用

```
1 from tqcenter import tq  
2  
3 tq.initialize(__file__)  
4  
5 sc_one = tq.get_scjy_value_by_date(field_list=['SC6', 'SC7', 'SC8', 'SC9', 'SC10'])  
6 print(sc_one)
```

py



数据样本

1

```
{'SC10': ['0.00', '181415.13'], 'SC6': ['-30479.00', '0.00'], 'SC7': ['-264
```

← [获取市场交易数据](#)`get_scjy_value`

[获取股票的单个数据\(非序列\)](#)`get_gp_one_data` →

获取股票的单个财务数据

get_gp_one_data

根据证券代码，获取股票的单个数据，需要先在客户端种下载股票数据包

```
1 get_gp_one_data(stock_list: List[str] = [],  
2                 field_list: List[str] = []) -> Dict:
```

py

输入参数

参数	是否必选	参数类型	参数说明
field_list	Y	List[str]	字段筛选，不能为空（如G047 表示是第47号个股数据最新业绩预告 本期扣非净利润预计同比增减幅上限%）这个值，GO为gp or 的首字母大写
stock_list	Y	List[str]	证券代码列表

输出参数

名称	类型	数值	说明
GO1	double		发行价(元)
GO2	double		总发行数量(万股)
GO3	double		一致预期目标价(元)[注：一致预期值均为近半年内各家机构预测数值的平均值]
GO4	double		一致预期T年度
GO5	double		一致预期T年每股收益



GO6	double		一致预期T+1年每股收益
GO7	double		一致预期T+2年每股收益
GO8	double		一致预期T年净利润(万元)
GO9	double		一致预期T+1年净利润(万元)
GO10	double		一致预期T+2年净利润(万元)
GO11	double		一致预期T年营业收入(万元)
GO12	double		一致预期T+1年营业收入(万元)
GO13	double		一致预期T+2年营业收入(万元)
GO14	double		一致预期T年营业利润(万元)
GO15	double		一致预期T+1年营业利润(万元)
GO16	double		一致预期T+2年营业利润(万元)
GO17	double		一致预期T年每股净资产(元)
GO18	double		一致预期T+1年每股净资产(元)
GO19	double		一致预期T+2年每股净资产(元)
GO20	double		一致预期T年净资产收益率(%)
GO21	double		一致预期T+1年净资产收益率(%)
GO22	double		一致预期T+2年净资产收益率(%)
GO23	double		一致预期T年PE
GO24	double		一致预期T+1年PE
GO25	double		一致预期T+2年PE
GO26	double		最新解禁日(YYMMDD格式)
GO27	double		最新解禁数量 (万股)
GO28	double		下一报告期的预约披露时间



GO29	double		最新持股机构家数
GO30	double		最新机构持股总量（万股）
GO31	double		最新持股基金家数
GO32	double		最新基金持股量（万股）
GO33	double		最新总股本（万股）
GO34	double		最新实际流通A股（万股）
GO35	double		最新业绩预告 报告期(YYMMDD格式)
GO36	double		最新业绩预告 本期归母净利润下限（万元）
GO37	double		最新业绩预告 本期归母净利润上限（万元）
GO38	double		最新业绩预告 本期归母净利润预计同比增减幅下限%
GO39	double		最新业绩预告 本期归母净利润预计同比增减幅上限%
GO40	double		最新业绩快报 报告期
GO41	double		最新业绩快报 归母净利润（万元）
GO42	double		分红募资 派现总额（万元）
GO43	double		分红募资 募资总额（万元）
GO44	double		最新业绩预告 本期扣非净利润下限(万元)
GO45	double		最新业绩预告 本期扣非净利润上限(万元)
GO46	double		最新业绩预告 本期扣非净利润预计同比增减幅下限%
GO47	double		最新业绩预告 本期扣非净利润预计同比增减幅上限%

接口使用

```
1 from tqcenter import tq
2
3 tq.initialize(__file__)
4
```

py



数据样本

1	{'688318.SH': {'G01': '107.41', 'G02': '1667.00', 'G03': '0.00', 'G04': '20
---	---

← [获取指定日期市场交易数据get_scjy_value_by_date](#)

[获取A股板块代码列表get_sector_list](#) →



很抱歉，您访问的网页出现内部错误，请稍后再试！

[返回首页](#)

获取系统成份股

根据入参返回指定证券代码列表

```
1 get_stock_list(market = None) -> List:
```

py

输入参数

参数	是否必选	参数类型	参数说明
market	Y	str	指定代码

```
1 默认为全部A股
2 0:自选股 1:持仓股
3 5:所有A股 6:上证指数成份股 7:上证主板 8:深证主板 9:重点指数
4 10:所有板块指数 11:缺省行业板块 12:概念板块 13:风格板块 14:地区板块 15:缺省
5 21:含H股 22:含可转债 23:沪深300 24:中证500 25:中证1000 26:国证2000 27:中证
6 30:REITs 31:ETF基金 32:可转债 33:LOF基金 34:所有可交易基金 35:所有沪深基金
7 49:金融类企业 50:沪深A股 51:创业板 52:科创板 53:北交所
8 101:国内期货 102:港股 103:美股
```

接口使用

```
1 from tqcenter import tq
2 tq.initialize(__file__)
3 stock_list = tq.get_stock_list('16')
4 print(stock_list)
```

py

数据样本



← [获取A股板块代码列表get_sector_list](#)

[获取板块成份股get_stock_list_in_sector](#) →

获取板块成份股get_stock_list_in_sector

根据板块代码获取其成份股列表

```
1 get_stock_list_in_sector(block_code: str,  
2                           block_type: int = 0) -> List:
```

py

输入参数

参数	是否必选	参数类型	参数说明
block_code	Y	str	板块代码
block_type	N	str	板块类型

- 获取A股成份股时支持板块名称或板块代码两种方式传入
- block_type=0 表示传入板块代码或名称（默认）
- block_type=1 表示传入自定义板块简称 需要是客户端中预先定义好板块简称 不能是 自选股 或 临时条件股

接口使用

```
1 from tqcenter import tq  
2 tq.initialize(__file__)  
3 #通过板块代码获取成份股  
4 block_stocks = tq.get_stock_list_in_sector('880081.SH')  
5 print(block_stocks)  
6 print(len(block_stocks))  
7  
8 #通过板块名获取成份股  
9 block_stocks = tq.get_stock_list_in_sector('钛金属')  
10 print(block_stocks)  
11 print(len(block_stocks))  
12  
13 #获取自定义板块成份股
```

py



```
10 print(len(block_stocks))
```

数据样本

```
1 ['159922.SZ', '510500.SH', '512500.SH']
2 3
3 ['000545.SZ', '000629.SZ', '000635.SZ', '000688.SZ', '000709.SZ', '000962.SZ',
4 23
5 ['600000.SH', '600004.SH', '600006.SH', '600007.SH', '600008.SH', '600009.SH',
6 7
```

← [获取系统成份股get_stock_list](#)

[获取自定义板块列表get_user_sector](#) →



很抱歉，您访问的网页出现内部错误，请稍后再试！

[返回首页](#)

创建自定义板块

在通达信客户端中创建自定义板块

```
1 create_sector(block_code:str = '',
2               block_name:str = ''):
```

py

输入参数

参数	是否必选	参数类型	参数说明
block_code	Y	str	自定义板块简称
block_name	Y	str	自定义板块名称

接口使用

```
1 from tqcenter import tq
2 tq.initialize(__file__)
3 create_ptr = tq.create_sector(block_code='CSBK2', block_name='测试板块2')
4 print(create_ptr)
```

py

数据样本

```
1 {
2     "Error" : "创建CSBK2板块成功",
3     "ErrorId" : "0",
4     "run_id" : "1"
5 }
```



删除自定义板块

删除通达信客户端中的自定义板块

```
1 delete_sector(block_code:str = '',
2               block_name:str = '');
```

py

输入参数

参数	是否必选	参数类型	参数说明
block_code	Y	str	自定义板块简称
block_name	Y	str	自定义板块名称

接口使用

```
1 from tqcenter import tq
2 tq.initialize(__file__)
3 delete_ptr = tq.delete_sector(block_code='CSBK', block_name='测试板块')
4 print(delete_ptr)
```

py

数据样本

```
1 {
2     "Error" : "删除CSBK2板块成功",
3     "ErrorId" : "0",
4     "run_id" : "1"
5 }
```



创建自定义板块

重命名通达信客户端中的自定义板块

```
1 rename_sector(block_code:str = '',  
2               block_name:str = ''):
```

py

输入参数

参数	是否必选	参数类型	参数说明
block_code	Y	str	自定义板块简称
block_name	Y	str	重命名后的自定义板块名称

接口使用

```
1 from tqcenter import tq  
2 tq.initialize(__file__)  
3 rename_ptr = tq.rename_sector(block_code='CSBK', block_name='测试板块重命名')  
4 print(rename_ptr)
```

py

数据样本

```
1 {  
2     "Error" : "重命名CSBK板块成功",  
3     "ErrorId" : "0",  
4     "run_id" : "1"  
5 }
```



添加自定义板块成份股

往指定自定义板块中添加成份股

```
1 send_user_block(block_code: str = '',  
2                 stocks: List[str] = [],  
3                 show: bool = False) -> Dict:
```

py

输入参数

参数	是否必选	参数类型	参数说明
block_code	Y	str	自定义板块简称
stocks	Y	List[str]	添加的自选股
show	N	str	客户端是否切换至对应板块界面

- block_code 为客户端已有的自定义板块简称，如果不存在则无效果，空则为添加到临时条件股
- block_code存在，传入空列表则表示清空该板块所有股票，否则为添加新股票

接口使用

```
1 from tqcenter import tq  
2 tq.initialize(__file__)  
3 zxcg_result = tq.send_user_block(block_code='CSBK', stocks=["600000.SH", "600
```

py

数据样本



← [重命名自定义板块rename_sector](#)

[清空自定义板块成份股clear_sector](#) →

清空自定义板块成份股

清空指定通达信客户端自定义板块的成份股

```
1 clear_sector(block_code:str = '',  
2             block_name:str = '');
```

py

输入参数

参数	是否必选	参数类型	参数说明
block_code	Y	str	自定义板块简称
block_name	Y	str	自定义板块名称

接口使用

```
1 from tqcenter import tq  
2 tq.initialize(__file__)  
3 clear_ptr = tq.clear_sector(block_code='CSBK', block_name='测试板块重命名')  
4 print(clear_ptr)
```

py

数据样本

```
1 {  
2     "Error" : "清空CSBK板块成功",  
3     "ErrorId" : "0",  
4     "run_id" : "1"  
5 }
```





很抱歉，您访问的网页出现内部错误，请稍后再试！

[返回首页](#)



期货品种基本信息 product_basic暂未开放

← [可转债基础信息get_cb_info](#)

[合约详细信息 future_basic](#)暂未开放 →



合约详细信息 future_basic暂未开放

← [期货品种基本信息 product_basic](#)暂未开放

[常量枚举](#) →

市场类型

名称	类型	数值	说明
.SZ	int	0	深圳交易所
.SH	int	1	上海证券交易所
.BJ	int	2	北京交易所
.NQ	int	44	新三板
.SHO	int	8	上海个股期权
.SZO	int	9	深圳个股期权
.HK	int	31	港股个股
.US	int	74	美国股票

dividend_type复权类型

名称	类型	数值	说明
type	str	none	不复权
type	str	front	前复权
type	str	back	后复权

period周期入参类型

名称	类型	数值	说明
period	str	1m	1分钟
period	str	5m	5分钟
period	str	15m	15分钟



period	str	30m	30分钟
period	str	1h	60分钟（1小时）
period	str	1d	1天
period	str	1w	1周
period	str	1mon	1月
period	str	1q	1季
period	str	1y	1年
period	str	tick	分笔

← [合约详细信息](#) `future_basic`暂未开放

[回测及模拟交易](#) →

什么是量化交易

量化交易是指利用计算机科技并采用一定的数学模型去实现投资理念、实现投资策略的过程。简单的说，量化交易主要是做这样的事：

**一个简单的投资想法 => 可执行的交易策略 => 可执行的代码程序 => 检验交易策略效果
=> 实盘交易验证改进**

Step 1: 从一个简单的投资想法开始

投资想法即我们认为可能会盈利的投资方法、理念，比如熊市时期银行股是潜力股、复制基金经理的增强指数、金叉买入死叉卖出等等。这些想法通常以网络、书本和讲座等为载体，来源于投顾、同行以及自己的经验总结等等。

以一个简单的投资想法为例：

- | | |
|---|--------------|
| 1 | 如果遇到股价金叉，则买入 |
| 2 | 如果遇到股价死叉，则卖出 |

py

Step 2: 完善这个想法，形成明确的可执行的交易策略

简单的投资想法通常比较模糊，我们需要将其细化成明确的可执行的策略，目的是为了能得到确定的信号进行交易操作。

一个可执行的交易策略至少需要明确以下几点：

1. Security：确定投资品种或范围
2. Condition：确定触发买/卖的具体条件
3. Quantity：确定买卖的数量/金额等

明确的可执行的交易策略的判断基准：根据交易策略的描述，不同的人在同一情形下，都能做出相同的交易操作。

上述关于金叉死叉的投资想法，显然它是不够明确的（可度量/可计算）。所以我们进一步细化：

现在，我们基本已经把之前的想法细化成了明确的可执行的交易策略。当然，可能还有些地方不够明确或者参数需要改动，这些可以随时想到随时修正，不必一次做到完美。

现在，我们想知道这样操作究竟会不会赚钱？

Step 3: 编写一段代码，把交易策略转成可执行的代码程序

为了验证这个策略是否赚钱，我们需要把明确后的交易策略通过编程转成程序，计算机能根据历史数据/实时数据执行该策略产生模拟交易，或者根据实时数据执行该策略产生实盘交易。

把上述策略翻译成计算机可以识别的代码语言，即类似这样的代码：

```
1
2 import pandas as pd
3 import vectorbt as vbt
4 from tqcenter import tq
5
6 tq.initialize(__file__)
7
8 # 解决 pandas future warning
9 pd.set_option('future.no_silent_downcasting', True)
10
11 # ===== 核心配置（用户可直接修改这里） =====
12 target_start = '20240930' # 【目标回测开始时间】（真正想回测的起始日）
13 target_end = '20250930' # 【目标回测结束时间】
14 stock_code_list = ['688318.SH'] # 股票代码
15 window = 5 # MA指标周期（如MA5、MA10、MA20，改这里自动适配历史数据）
16 # =====
17
18 start_time = (pd.to_datetime(target_start) - pd.Timedelta(days=window + 10))
19
20 # 1. 获取价格数据
21 df_real = tq.get_market_data(
22     field_list=['Close', 'Open'],
23     stock_list=stock_code_list,
24     start_time=start_time,
25     end_time=target_end,
```



```
28     fill_data=True
29 )
30 close_df = tq.price_df(df_real, 'Close', column_names=stock_code_list)
31 open_df = tq.price_df(df_real, 'Open', column_names=stock_code_list)
32
33
34 # 2. 买卖信号计算与生成
35 ma5_dynamic = vbt.MA.run(close_df, window=window).ma
36 ma5_dynamic.columns = close_df.columns
37
38 entries_raw = close_df.vbt.crossed_above(ma5_dynamic)
39 exits_raw = close_df.vbt.crossed_below(ma5_dynamic)
40
41 # 信号移位+1
42 entries_df = entries_raw.shift(1).fillna(False).astype(bool)
43 exits_df = exits_raw.shift(1).fillna(False).astype(bool)
44
45
46 # 3. 执行回测
47 portfolio = vbt.Portfolio.from_signals(
48     close=close_df,          # 净值计算用未复权收盘价
49     entries=entries_df,      # 延迟后的买入信号
50     exits=exits_df,          # 延迟后的卖出信号
51     price=open_df,           # 含滑点的成交价格
52     init_cash=100000,        # 初始资金10万元
53     fees=0.0003,             # 手续费0.03% (双边)
54     freq='D',                # 日线频率
55     size_granularity=100     # A股最小交易单位100股
56 )
57
58
59 # 4. 输出回测结果
60 print(f"\n=====投资组合回测表现=====")
61 print(portfolio.stats())
62 print(f"\n=====投资组合回测记录=====")
63 print(portfolio.trades.records_readable)
64
```

这样一来，刚才细化好的策略转成了代码，计算机就能理解并执行了。

Step 4: 回测或者模拟交易，检验策略效果

****回测**是让计算机能根据一段时间区间内的历史的数据来模拟执行该策略，根据结果评价并改进策略。****如果结果不好，则需要分析原因并改进。如果结果不错，则可以考虑用模拟交易进一步验证。**

****模拟交易**是让计算机能根据未来的实际数据模拟执行该策略一段时间区间，根据结果评价并改进策略。****如果策略在回测与模拟交易的表现都非常好，我们可以考虑进行完全真金白银的实盘交易。**

回测举例说明：

1. 策略环境：设定初始虚拟资产100万元；选择一段历史时间区间：20100101到20200101；把该时间区间的各种数据如收盘股价行情等发给计算机。
2. 策略执行：计算机利用这些数据模仿历史真实的市场，执行我们编写的策略程序。
3. 策略评估：计算机会出具一份报告，根据这个报告我们知道，在20100101期初的100万元，按照我们的策略交易到期末20200101，会怎样？一般包括盈亏情况，下单情况，持仓变化，以及一些统计指标等，根据此评估交易策略的好坏。

模拟交易举例说明：

1. 策略环境：设定初始的虚拟资产比如100万元，选择开始执行模拟交易的时间点，比如下周一。那么从下周一开始，股市开始交易，真实的行情数据就会实时地发送到计算机。
2. 策略执行：计算机利用真实的数据模仿真实的市场，执行你的策略代码输出买卖队列，模拟系统会记录每一笔买卖记录。
3. 策略评估：我们可以得到一份实时更新的策略评估报告，这报告类似于回测得到的报告，不同的是会根据实际行情变化更新；同样我们能据此评估交易策略的好坏。

Step 5：实盘执行交易策略，并持续优化改进策略

实盘交易就是让计算机能根据实际行情，用真实资金账号来自动下单交易。**注意，这时不再是用虚拟资产进行模拟交易，实盘交易账户上的盈亏都是真金白银。**

实盘交易一般也会给出一份类似模拟交易的投资分析报告，通过实时观察策略的实盘表现、根据投资理念的变化、市场状况的变化及时修正、改善和优化策略，使之保持持续盈利能力。





很抱歉，您访问的网页出现内部错误，请稍后再试！

[返回首页](#)

订阅行情涨幅突破实时预计

第一步：设置预警条件，并发送预警结果到客户端

```
1      #订阅板块成分股行情，涨幅突破实时预警，首次突破后取消该证券行情订阅监控
2      import json
3      import time
4      import signal
5      import sys
6      from datetime import datetime, timedelta
7      from collections import defaultdict
8      from tqcenter import tq
9
10     # ===== 全局配置 =====
11     # 板块配置：支持多个板块/自定义板块
12     SECTOR_NAMES = ['通达信88'] # 可扩展为其他板块名称或代码
13     PRICE_RISE_THRESHOLD = 5.0 # 涨幅阈值>5%
14     ANTI_SHAKE_SECONDS = 10 # 防抖间隔
15     BATCH_SUBSCRIBE_SIZE = 50 # 分批订阅大小（避免单次订阅过多）
16
17     # 全局变量
18     SUBSCRIBE_CODES = [] # 动态获取的监控股票代码列表
19     last_warn_time = defaultdict(int)
20     EXIT_FLAG = False
21     TRIGGERED_STOCKS = set() # 记录已首次触发预警的股票（避免重复监控/推送）
22
23     # ===== 信号处理函数 =====
24     def signal_handler(signum, frame):
25         """处理Ctrl+C (SIGINT) 信号"""
26         global EXIT_FLAG
27         print(f"\n\n[{datetime.now().strftime('%H:%M:%S')}] 接收到退出信号 (Ctrl+C)")
28         EXIT_FLAG = True
29         # 强制取消订阅+关闭TDX
30         try:
31             unsubscribe_stocks()
32         except Exception as e:
33             print(f"取消订阅失败: {e}")
34
35         print("资源清理完成，程序退出！")
36         sys.exit(0)
```



```
39 def get_valid_stock_codes(sector_names):
40     """
41     从指定板块获取有效股票代码列表
42     :param sector_names: 板块名称列表
43     :return: 去重后的有效股票代码列表
44     """
45     valid_codes = set() # 用集合去重
46     for sector in sector_names:
47         try:
48             # 获取板块股票列表 (TDX初始化后调用)
49             sector_codes = tq.get_stock_list_in_sector(sector)
50             if not sector_codes:
51                 print(f"[{datetime.now().strftime('%H:%M:%S')}] 警告: 板块{s}无股票" % sector)
52                 continue
53
54             # 过滤无效代码 (空值、格式错误)
55             for code in sector_codes:
56                 if code and isinstance(code, str) and (code.endswith('.SH') or code.endswith('.SZ')):
57                     valid_codes.add(code)
58                 else:
59                     print(f"[{datetime.now().strftime('%H:%M:%S')}] 过滤无效代码: {code}")
60
61         except Exception as e:
62             print(f"[{datetime.now().strftime('%H:%M:%S')}] 获取板块{sector}股票失败: {e}")
63             import traceback
64             traceback.print_exc()
65
66     # 转为列表并排序
67     valid_codes_list = sorted(list(valid_codes))
68     print(f"[{datetime.now().strftime('%H:%M:%S')}] 从板块{sector_names}获取有效股票代码列表")
69     return valid_codes_list
70
71 def batch_subscribe(stocks, batch_size):
72     """
73     分批订阅股票 (避免单次订阅过多)
74     :param stocks: 股票列表
75     :param batch_size: 每批订阅数量
76     :return: 整体订阅结果 (True/False)
77     """
78     total_success = True
79     for i in range(0, len(stocks), batch_size):
80         batch = stocks[i:i+batch_size]
81         try:
```



```
84         if not sub_res:
85             print(f"[{datetime.now().strftime('%H:%M:%S')}] 第{i//batch_size+1}批失败")
86             total_success = False
87         else:
88             print(f"[{datetime.now().strftime('%H:%M:%S')}] 第{i//batch_size+1}批成功")
89     except Exception as e:
90         print(f"[{datetime.now().strftime('%H:%M:%S')}] 第{i//batch_size+1}批失败")
91         total_success = False
92     return total_success
93
94 def unsubscribe_single_stock(stock_code):
95     """
96     取消单只股票的订阅（首次触发后不再监控）
97     :param stock_code: 股票代码
98     :return: 取消结果（True/False）
99     """
100     try:
101         un_sub_res = tq.unsubscribe_hq(stock_list=[stock_code])
102         if un_sub_res:
103             # 从全局订阅列表中移除
104             if stock_code in SUBSCRIBE_CODES:
105                 SUBSCRIBE_CODES.remove(stock_code)
106             return True
107         return False
108     except Exception as e:
109         print(f"[{datetime.now().strftime('%H:%M:%S')}] 取消{stock_code}订阅失败")
110         return False
111
112 # ===== 核心回调函数 =====
113 def price_rise_callback(data_str):
114     try:
115         code_json = json.loads(data_str)
116         code = code_json.get('Code')
117
118         # 前置过滤：无效数据/非监控股票/已触发过的股票（直接返回，不输出日志）
119         if (code_json.get('ErrorId') != "0" or not code) or \
120             (code not in SUBSCRIBE_CODES) or \
121             (code in TRIGGERED_STOCKS):
122             return
123
124         # 获取最新行情数据
125         report_ptr = tq.get_full_tick(code)
126
```



```
129
130     if report_ptr:
131         latest_price = round(float(report_ptr['Now']), 2)
132         pre_close = round(float(report_ptr['LastClose']), 2)
133
134         if pre_close <= 0 and latest_price > 0:
135             pre_close = latest_price - 0.01
136
137     # 过滤最新价/昨收价无效的情况
138     if latest_price <= 0 or pre_close <= 0:
139         return
140
141     # 计算涨幅
142     rise_rate = round(((latest_price - pre_close) / pre_close) * 100, 2)
143
144     # 仅处理满足涨幅阈值+防抖的情况
145     if rise_rate > PRICE_RISE_THRESHOLD:
146         current_time = int(time.time())
147         if current_time - last_warn_time[code] < ANTI_SHAKE_SECONDS:
148             return
149
150         # 标记为已触发, 后续不再处理
151         TRIGGERED_STOCKS.add(code)
152         last_warn_time[code] = current_time
153
154         # 取消该股票的订阅 (不再监控)
155         unsubscribe_single_stock(code)
156
157         # 发送预警
158         warn_time = datetime.now().strftime("%Y%m%d%H%M%S")
159         reason = (
160             f"涨幅突破"
161         )
162
163         try:
164             # 成交量用实际值, 无则填0
165             volume = report_ptr.get('Volume', '0') if report_ptr else 0
166             warn_res = tq.send_warn(
167                 stock_list=[code],
168                 time_list=[warn_time],
169                 price_list=[str(latest_price)],
170                 close_list=[str(pre_close)],
171                 volum_list=[volume],
```



```
174         reason_list=[reason],
175         count=1
176     )
177     print(f"[{datetime.now().strftime('%H:%M:%S')}] {reason}")
178     print(f"[{datetime.now().strftime('%H:%M:%S')}] 预警发送结果")
179     print(f"[{datetime.now().strftime('%H:%M:%S')}] 已取消{code}")
180 except Exception as e:
181     print(f"\n[{datetime.now().strftime('%H:%M:%S')}] {code} 发
182
183 except Exception as e:
184     print(f"\n[{datetime.now().strftime('%H:%M:%S')}] 回调函数执行异常:
185     import traceback
186     traceback.print_exc()
187
188     return None
189
190 # ===== 订阅/取消订阅函数=====
191 def subscribe_stocks():
192     """订阅股票 (分批订阅+容错) """
193     if not SUBSCRIBE_CODES:
194         print(f"\n[{datetime.now().strftime('%H:%M:%S')}] 无有效股票可订阅,
195         return False
196
197     print(f"\n开始批量订阅股票 (总计{len(SUBSCRIBE_CODES)}只) ")
198     sub_result = batch_subscribe(SUBSCRIBE_CODES, BATCH_SUBSCRIBE_SIZE)
199     print(f"批量订阅最终结果: {'成功' if sub_result else '部分/全部失败'}")
200     return sub_result
201
202 def unsubscribe_stocks():
203     """取消订阅 (分批取消) """
204     if not SUBSCRIBE_CODES:
205         print(f"\n[{datetime.now().strftime('%H:%M:%S')}] 无已订阅股票, 跳过
206         return False
207
208     print(f"\n开始批量取消订阅股票 (总计{len(SUBSCRIBE_CODES)}只) ")
209     total_success = True
210     for i in range(0, len(SUBSCRIBE_CODES), BATCH_SUBSCRIBE_SIZE):
211         batch = SUBSCRIBE_CODES[i:i+BATCH_SUBSCRIBE_SIZE]
212         try:
213             print(f"取消第{i//BATCH_SUBSCRIBE_SIZE + 1}批订阅: {batch[:5]}..
214             un_sub_ptr = tq.unsubscribe_hq(stock_list=batch)
215             if not un_sub_ptr:
216                 print(f"第{i//BATCH_SUBSCRIBE_SIZE + 1}批取消失败: {un_sub_p
```

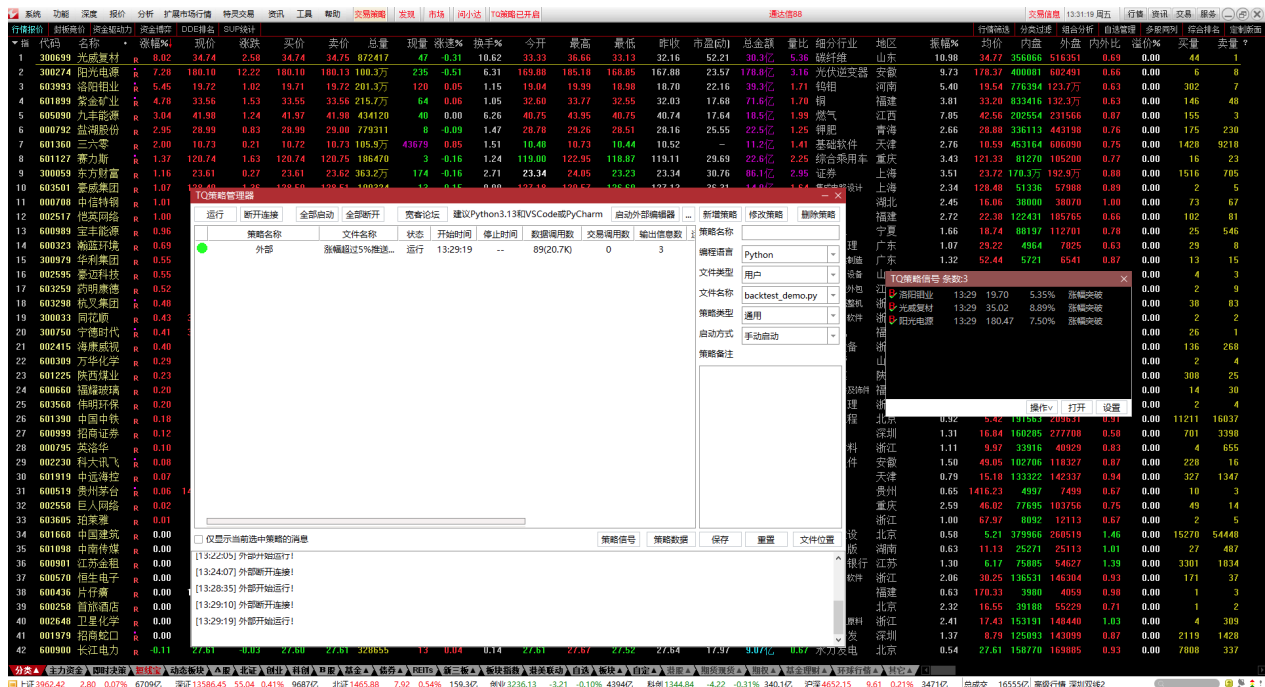



```
219         print(f"第{1}/{BATCH_SUBSCRIBE_SIZE} 批取消异常: {e}")
220         total_success = False
221     print(f"批量取消订阅最终结果: {'成功' if total_success else '部分/全部失败'}")
222     return total_success
223
224     # ===== 主程序 =====
225     if __name__ == "__main__":
226         # 注册SIGINT信号处理 (优先于默认的KeyboardInterrupt)
227         signal.signal(signal.SIGINT, signal_handler)
228
229         # 1. 初始化TDX (仅执行1次, 无重试)
230         try:
231             tq.initialize(__file__)
232             print(f"程序启动时间: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
233             print(f"TDX初始化成功")
234         except Exception as e:
235             print(f"TDX初始化失败: {e}")
236             exit(1)
237
238         # 2. 获取板块股票列表
239         SUBSCRIBE_CODES = get_valid_stock_codes(SECTOR_NAMES)
240         if not SUBSCRIBE_CODES:
241             print("未获取到任何有效股票代码, 程序退出")
242             exit(1)
243
244         # 3. 订阅股票
245         subscribe_stocks()
246
247         # 4. 运行提示
248         print(f"\n=== 涨幅监控启动 ===")
249         print(f"监控板块: {SECTOR_NAMES}")
250         print(f"监控股票总数: {len(SUBSCRIBE_CODES)}")
251         print(f"涨幅阈值: >{PRICE_RISE_THRESHOLD}%")
252         print(f"防抖间隔: {ANTI_SHAKE_SECONDS}秒")
253         print(f"分批订阅大小: {BATCH_SUBSCRIBE_SIZE}只/批")
254         print("按 Ctrl+C 退出程序...\n")
255
256         # 5. 通过全局标记控制退出
257         try:
258             while not EXIT_FLAG:
259                 time.sleep(0.1)
260         except Exception as e:
261             print(f"主循环异常: {e}")
```



第二步：打开通达信金融终端查看运行结果

通达信金融终端



← 执行选股入板块

计算调仓信号并快速买卖 →

计算调仓信号并快速买卖

第一步：计算信号并发送预警，以

```
1 from datetime import datetime, timedelta
2 from tqcenter import tq as tdxdata
3 import vectorbt as vbt
4 import pandas as pd
5
6 # 初始化
7 tdxdata.initialize(__file__)
8 run_time = datetime.now()
9 run_time_str = run_time.strftime("%Y-%m-%d %H:%M:%S")
10 # 预警时间戳 (格式: YYYYMMDDHHMMSS)
11 warn_time = run_time.strftime("%Y%m%d%H%M%S")
12
13 # ===== 1. 配置参数 =====
14 N = 5 # 均线周期
15 batch_codes = tdxdata.get_stock_list_in_sector('通达信88')
16 end_date = run_time.strftime("%Y%m%d")
17 start_date = (run_time - timedelta(days=2 * N + 20)).strftime("%Y%m%d")
18
19 # ===== 2. 获取并处理数据 =====
20 # 获取日线Close数据 (保留完整索引用于日期筛选)
21 df_real = tdxdata.get_market_data(
22     field_list=['Close'],
23     stock_list=batch_codes,
24     start_time=start_date,
25     end_time=end_date,
26     dividend_type='front',
27     period='1d',
28     fill_data=True
29 )
30 close_df = tdxdata.price_df(df_real, 'Close', column_names=batch_codes)
31
32 # 计算均线+生成信号
33 ma = vbt.MA.run(close_df, window=N).ma
34 ma.columns = close_df.columns
35 entries = close_df.vbt.crossed_above(ma) # 上穿 (买入)
36 exits = close_df.vbt.crossed_below(ma) # 下穿 (卖出)
```



```
39 prev_date = close_df.index[-2] if len(close_df.index) >= 2 else latest_date
40
41 # ===== 3. 筛选最新买卖信号 =====
42 buy_signals = {}
43 sell_signals = {}
44
45 # 遍历股票筛选信号
46 for code in batch_codes:
47     # 确保股票有足够的交易数据
48     if code not in close_df.columns:
49         continue
50
51     # 今日收盘价
52     today_close = close_df.loc[latest_date, code]
53     # 上一个工作日收盘价
54     prev_close = close_df.loc[prev_date, code] if len(close_df.index) >= 2
55
56     # 买入信号：最新日期Close上穿均线
57     if entries.loc[latest_date, code]:
58         buy_signals[code] = {
59             'today_close': round(today_close, 2),    # 今日close
60             'prev_close': round(prev_close, 2),     # 上一个工作日close
61             'ma_price': round(ma.loc[latest_date, code], 2)
62         }
63     # 卖出信号：最新日期Close下穿均线
64     if exits.loc[latest_date, code]:
65         sell_signals[code] = {
66             'today_close': round(today_close, 2),    # 今日close
67             'prev_close': round(prev_close, 2),     # 上一个工作日close
68             'ma_price': round(ma.loc[latest_date, code], 2)
69         }
70
71 # ===== 4. 生成并发送MSG =====
72 def send_msg(content):
73     msg = f"MSG,{content}"
74     print(msg)
75     try:
76         tdxdata.send_message(msg)
77     except Exception as e:
78         print(f"发送失败: {e}")
79
80 # 统计行
81 stat_line = (
```



```
84
85
86 print("\n=== MSG格式 (TQ策略管理器显示区域) ===")
87 send_msg(stat_line)
88
89 # 处理买入信号
90 if buy_signals:
91     send_msg(f"=== 买入信号 (Close上穿{N}日均线) ===")
92     for idx, (code, info) in enumerate(buy_signals.items(), 1):
93         line = f"{idx}. {code}: 买入信号, 今日Close:{info['today_close']}, 昨"
94         send_msg(line)
95
96 # 处理卖出信号
97 if sell_signals:
98     send_msg(f"=== 卖出信号 (Close下穿{N}日均线) ===")
99     for idx, (code, info) in enumerate(sell_signals.items(), 1):
100         line = f"{idx}. {code}: 卖出信号, 今日Close:{info['today_close']}, 昨"
101         send_msg(line)
102
103 # 无信号的情况
104 if not buy_signals and not sell_signals:
105     send_msg(f"运行时间: {run_time_str}, 均线周期: {N}天, 无买入或卖出信号")
106
107 # ===== 5. 调用send_warn接口发送预警 =====
108 def send_trade_warn():
109     """发送买卖信号对应的预警 (精简版, 仅保留核心逻辑) """
110     # 合并所有信号用于发送预警
111     all_signals = []
112     if buy_signals:
113         all_signals.extend([(code, info, '买入') for code, info in buy_signals.items()])
114     if sell_signals:
115         all_signals.extend([(code, info, '卖出') for code, info in sell_signals.items()])
116
117     if not all_signals:
118         print("\n无预警信息需要发送")
119         return
120
121     # 构造预警参数列表
122     codes = []
123     time_list = []
124     price_list = [] # 今日close
125     close_list = [] # 上一个工作日close
126     volum_list = []
```



```
129     reason_list.append('')
130
131     for code, info, trade_type in all_signals:
132         codes.append(code)
133         time_list.append(warn_time)
134         price_list.append(str(info['today_close'])) # 替换为今日close
135         close_list.append(str(info['prev_close'])) # 替换为上一个工作日c
136         volum_list.append('0')
137         bs_flag_list.append('0' if trade_type == '买入' else '1')
138         warn_type_list.append('1')
139         reason_list.append(f"{trade_type}信号")
140
141     # 调用预警接口
142     try:
143         warn_res = tdxdata.send_warn(
144             stock_list=codes,
145             time_list=time_list,
146             price_list=price_list,
147             close_list=close_list,
148             volum_list=volum_list,
149             bs_flag_list=bs_flag_list,
150             warn_type_list=warn_type_list,
151             reason_list=reason_list,
152             count=len(codes)
153         )
154         print(f"\n预警发送完成, 共发送 {len(codes)} 条预警, 接口返回: {warn_re
155     except Exception as e:
156         print(f"\n预警发送失败: {e}")
157
158     # 执行预警发送
159     send_trade_warn()
160
161     print("\n所有消息发送完成! ")
162     tdxdata.close()
163
```

第二步:双击TQ策略信号, 快速打开闪电买卖, 根据输出的买/卖信号打开买/卖界面



策略名称	文件名称	状态	开始时间	停止时间	数据调用数	交易调用数	输出信息数	策略名称
Outside	均线买卖信号_send_w...	停止	09:08:45	09:08:46	89(76.1K)	0	11	编程语言 Python

TQ策略信号 条数:7

纽威股份	09:08	51.71	0.00%	卖出信号
豪威集团	09:08	126.92	0.00%	卖出信号
东方财富	09:08	23.35	0.00%	卖出信号
豪迈科技	09:08	83.96	0.00%	卖出信号
陕西煤业	09:08	21.68	0.00%	买入信号
小商品城	09:08	16.29	0.00%	买入信号
分众传媒	09:08	7.19	0.00%	买入信号

操作v 打开 设置

1.闪电买入

当前用户 模拟操盘(普)-86 113**42

股票代码 深A B0001350442

证券代码 002027 分众传媒

报价方式 限价委托

买入价格 7.19 未锁 元

最大可买 300 全部 股

买入数量 0 股

可买量 1/2 1/3 1/4 1/5

B->S 买入

预估金额:0.00 (0股)

☒ 仅显示当前选中策略的消息

策略信号 策略数据 保存 重置 文件位置

[09:08:46] MSG,1. 002595.SZ: 卖出信号, 今日Close:83.96, 昨日Close:83.96
[09:08:46] MSG,2. 300059.SZ: 卖出信号, 今日Close:23.35, 昨日Close:23.35
[09:08:46] MSG,3. 603501.SH: 卖出信号, 今日Close:126.92, 昨日Close:126.92
[09:08:46] MSG,4. 603699.SH: 卖出信号, 今日Close:51.71, 昨日Close:51.71
[09:08:46] Outside断开连接.

← [订阅行情涨幅突破实时预警](#)

[常见问题](#) →



Q: 运行的python文件可不可以随便放, 不一定在PYPlugins\user目录下?

A: 可以。在import tqcenter前添加通达信安装目录\PYPlugins\user这个绝对路径。

```
1 import sys
2 sys.path.append('C:/new_tdx64/PYPlugins/user')
3 from tqcenter import tq
4 tq.initialize(__file__)
```

py

← [计算调仓信号并快速买卖](#)